Course: Python Scientifique

Ali ZAINOUL <ali.zainoul.az@gmail.com>

CrystalClearCode
March 23, 2026

1 Introduction
- Introduction
- Installer Python et les librairies scientifiques
- Les notebooks Jupyter
- Environnements de développement intégrés
- Éditeur de texte VS Code

2 Tableaux NumPy
- Introduction
- Création d'un tableau NumPy
  - À partir d'une liste Python
- En utilisant différents types de constructeurs
  - La fonction `arange()`
  - La fonction `linspace()`
  - La fonction `ones()`
  - La fonction `eye()`
  - La fonction `full()`
  - Constructeurs de nombres aléatoires
  - La fonction `random.rand()`

- La fonction `random.randn()`
- La fonction `random.arange()`
- La fonction `random.randint()`
- Dupliquer ou créer une vue d'un tableau existant

- Les attributs de base d'un tableau NumPy
  - Dimensions
  - L'attribut `ndim`
  - L'attribut `size`
  - L'attribut `shape`
  - L'attribut `dtype`
- Certains usages des tableaux NumPy
  - Manipulation des dimensions
  - La fonction `reshape()`
  - La fonction `squeeze()`
  - La fonction `ravel()`
  - La fonction `flatten()`
  - Fusion des tableaux
  - La fonction `vstack()`
  - La fonction `hstack()`
  - La fonction `concatenate()`

- Opérations sur les tableaux NumPy
  - Indexation
  - Découpage/Extraction
  - Diffusion (*Broadcasting*)
  - Sélection/Masquage
  - Indexation booléenne

3 Pandas

- Introduction
- Structures de données
  - Séries (Series)
  - Tableau de données (DataFrame)
- Import et export de données
  - Lecture de données
  - Export de données
- Indexation et sélection
  - Sélection basique
  - Sélection avec loc et iloc
  - Techniques avancées
- Opérations et transformations
  - Opérations simples

- Agrégation et groupement
- Gestion des données manquantes
  - Détection
  - Suppression
  - Remplissage
- Traitement avancé
  - Fusion et jointures
  - Pivot et restructuration
  - Apply, map et transformations
- Index multi-niveaux
  - Création de MultiIndex
  - Manipulation de MultiIndex
- Manipulation de texte
  - Opérations de base
  - Expressions régulières
- Données temporelles
  - Création et parsing
  - Resampling et agrégation
  - Opérations avancées
- Performance et données massives

- Optimisation mémoire
- Formats alternatifs
- Alternatives à Pandas

4 Matplotlib

- Introduction
  - Le monde des librairies de visualisation
- Graphiques de base
  - Graphique linéaire
  - Nuage de points
  - Histogramme
- Graphiques multiples
  - Subplots
  - GridSpec
- Graphiques 3D
  - Configuration 3D
  - Scatter 3D et surfaces
- Interactivité
  - Interactivité dans Jupyter
  - Événements et animations

- Personnalisation
  - Graduations et axes
  - Limites et échelles
- Intégration avec Pandas
  - Plotting avec Pandas
  - Visualisations avancées

5 SciPy

- Introduction
  - Présentation
  - Modules principaux
- Intégration numérique
  - Intégration simple
  - Intégration multiple
- Optimisation
  - Minimisation de fonctions
  - Recherche de racines
  - Optimisation avec contraintes
- Algèbre linéaire
  - Systèmes linéaires
  - Décompositions matricielles

- Valeurs et vecteurs propres
- Statistiques
  - Distributions de probabilité
  - Tests statistiques
  - Analyse descriptive
- Transformée de Fourier
  - FFT et IFFT
  - Analyse spectrale
- Interpolation
  - Interpolation 1D
  - Interpolation 2D
- Ajustement de courbe
  - Modèles linéaires
  - Modèles non linéaires
  - Évaluation de la qualité
- Traitement du signal
  - Filtrage
  - Détection de pics
  - Analyse temps-fréquence
- Équations différentielles

- EDO simples
- Systèmes d'EDO
- Événements et terminaison

- Matrices creuses
  - Formats et stockage
  - Opérations
  - Applications

** Introduction **

Introduction à la formation

- Cette formation couvre les bibliothèques fondamentales du calcul scientifique en Python.
- Elle inclut **NumPy**, **Pandas**, **Matplotlib** et **SciPy**.
- L'objectif est de maîtriser la manipulation, l'analyse et la visualisation de données.
- Cette formation s'adresse aux développeurs, scientifiques et analystes de données.

Installation de Python et des librairies

- Installer Python depuis le site officiel : python.org.
- Utiliser `pip` pour installer les librairies :

```
pip install numpy pandas matplotlib scipy
```

- Alternative recommandée : distribution Anaconda.
- Vérification de l'installation :

```python
import numpy
import pandas
import matplotlib
import scipy
```

Introduction aux notebooks Jupyter

- Les notebooks Jupyter permettent d'exécuter du code Python de manière interactive.
- Ils combinent code, texte, équations et visualisations.
- Installation :

```
pip install notebook
```

- Lancement :

```
jupyter notebook
```

- Très utilisés en data science et en enseignement.

Exemple avec Jupyter

- Exemple de cellule de code :

```python
x = 10
y = 20
print(x + y)
```

- Exemple de cellule Markdown :
  - Permet d'écrire du texte formaté.
  - Supporte LaTeX pour les équations :

$$E = mc^2$$

IDE (Environnements de développement)

- Un IDE facilite le développement logiciel.
- Il fournit :
  - Auto-complétion
  - Debugging
  - Gestion de projet
- Exemples d'IDE :
  - PyCharm
  - Spyder
  - VS Code

Visual Studio Code

- VS Code est un éditeur de code léger et puissant.
- Supporte Python via des extensions.
- Installation de l'extension Python.
- Fonctionnalités principales :
  - Exécution de scripts Python
  - Debugger intégré
  - Support des notebooks

```python
print("Hello, VS Code!")
```

** NumPy **

Introduction à NumPy

- La bibliothèque NumPy permet d'effectuer des calculs numériques avec Python.
- NumPy est une bibliothèque Python utilisée pour travailler avec des **tableaux multidimensionnels**.
- Elle offre une grande variété de **fonctions** et de **méthodes** pour manipuler efficacement ces tableaux.
- NumPy est largement utilisé en science des données, en apprentissage automatique, en traitement du signal, et dans d'autres domaines où la manipulation de données est fréquente.

Installation de NumPy

- L'installation de NumPy se fait généralement via `pip`, le gestionnaire de paquets Python. Il est recommandé d'installer NumPy dans un environnement virtuel pour éviter les conflits avec d'autres bibliothèques Python.
- Pour importer NumPy dans un script Python, utilisez la commande `import numpy as np`. Cela permet d'accéder aux fonctions et méthodes de NumPy en utilisant l'alias `np`.

Exemple introductif à NumPy

- Importation de NumPy :

```python
import numpy as np
```

- Création d'un tableau NumPy à partir d'une liste :

```python
my_list = [1, 2, 3, 4, 5]
my_array = np.array(my_list)
```

- Affichage du tableau :

```python
print(my_array)
```

- Opération sur le tableau :

```python
my_array_squared = my_array ** 2
print(my_array_squared)
```

Création d'un tableau NumPy à partir d'une liste Python

- Pour créer un tableau NumPy à partir d'une liste Python, vous pouvez utiliser la fonction `numpy.array()`.
- Cette fonction prend une liste en entrée et retourne un tableau NumPy contenant les mêmes éléments que la liste.
- Voici un exemple de création d'un tableau NumPy à partir d'une liste Python :

```python
import numpy as np

# A Python List
my_list = [1, 2, 3, 4, 5]

# NumPy array creation from list
my_array = np.array(my_list)

print(my_array)
```

Exemple détaillé de création d'un tableau NumPy à partir d'une liste Python

- Importation de NumPy :

```python
import numpy as np
```

- Déclaration d'une liste Python :

```python
my_list = [1, 2, 3, 4, 5]
# Equivalent to : my_list = list([1,2,3,4,5])
```

- Création d'un tableau NumPy à partir de la liste :

```python
my_array = np.array(my_list)
```

- Affichage du tableau NumPy créé :

```python
print(my_array)
```

La fonction `arange()`

- La fonction `arange()` de NumPy permet de créer un tableau NumPy contenant une séquence d'éléments.
- Cette fonction est similaire à la fonction `range()` en Python standard.
- Voici un exemple d'utilisation de la fonction `arange()` :

```python
import numpy as np

# Creation of a NumPy array with function arange()
my_array = np.arange(0, 10, 2)

print(my_array)
```

La fonction `linspace()`

- La fonction `linspace()` de NumPy permet de créer un tableau NumPy avec une séquence d'éléments répartis uniformément entre deux valeurs.
- Elle prend en entrée les valeurs de début et de fin de la séquence, ainsi que le nombre d'éléments souhaités.
- Voici un exemple d'utilisation de la fonction `linspace()` :

```python
import numpy as np

# Création d'un tableau NumPy avec la fonction linspace
my_array = np.linspace(0, 10, 5)

print(my_array)
```

La fonction `ones()`

- La fonction `ones()` de NumPy permet de créer un tableau NumPy rempli de la valeur 1.
- Elle prend en entrée une forme (shape) sous forme de tuple pour spécifier les dimensions du tableau.
- Voici un exemple d'utilisation de la fonction `ones()` :

```python
import numpy as np

# Création d'un tableau NumPy rempli de 1
my_array = np.ones((3, 4))

print(my_array)
```

La fonction `eye()`

- La fonction `eye()` de NumPy permet de créer une matrice d'identité.
- Elle prend en entrée la taille de la matrice (nombre de lignes et de colonnes) ainsi que le type des éléments.
- Voici un exemple d'utilisation de la fonction `eye()` :

```python
import numpy as np

# Création d'une matrice d'identité 3x3
my_array = np.eye(3)

print(my_array)
```

La fonction `full()`

- La fonction `full()` de NumPy permet de créer un tableau NumPy rempli d'une valeur spécifiée.
- Elle prend en entrée une forme (shape) sous forme de tuple pour spécifier les dimensions du tableau, ainsi que la valeur à remplir.
- Voici un exemple d'utilisation de la fonction `full()` :

```python
import numpy as np

# Création d'un tableau NumPy rempli de la valeur 5
my_array = np.full((2, 3), 5)

print(my_array)
```

La fonction `random.rand()`

- La fonction `random.rand()` de NumPy permet de générer un tableau NumPy de nombres aléatoires à partir d'une distribution uniforme entre 0 et 1.
- Elle prend en entrée les dimensions souhaitées du tableau.
- Voici un exemple d'utilisation de la fonction `random.rand()` :

```python
import numpy as np

# Génération d'un tableau NumPy de dimensions (3, 4) avec des nombres aléatoires
random_array = np.random.rand(3, 4)

print(random_array)
```

La fonction `random.randn()`

- La fonction `random.randn()` de NumPy permet de générer un tableau NumPy de nombres aléatoires à partir d'une distribution normale (gaussienne) avec une moyenne de 0 et un écart-type de 1.
- Elle prend en entrée les dimensions souhaitées du tableau.
- Voici un exemple d'utilisation de la fonction `random.randn()` :

```python
import numpy as np

# Génération d'un tableau NumPy de dimensions (3, 4) avec des nombres aléatoires distribués selon une
    distribution normale
random_array = np.random.randn(3, 4)

print(random_array)
```

La fonction `random.arange()`

- La fonction `random.arange()` de NumPy permet de générer un tableau NumPy de nombres séquentiels dans une plage donnée avec un pas spécifié.
- Elle prend en entrée la valeur de début, la valeur de fin (non incluse) et le pas de la séquence.
- Voici un exemple d'utilisation de la fonction `random.arange()` :

```python
import numpy as np

# Génération d'un tableau NumPy de nombres séquentiels avec la fonction arange
sequential_array = np.arange(0, 10, 2)

print(sequential_array)
```

La fonction `random.randint()`

- La fonction `random.randint()` de NumPy permet de générer un tableau NumPy de nombres entiers aléatoires dans une plage donnée.
- Elle prend en entrée la valeur minimale (incluse), la valeur maximale (exclue) et les dimensions souhaitées du tableau.
- Voici un exemple d'utilisation de la fonction `random.randint()` :

```python
import numpy as np

# Génération d'un tableau NumPy avec des nombres entiers aléatoires dans une plage donnée
random_array = np.random.randint(1, 10, (3, 4))

print(random_array)
```

Dupliquer ou créer une vue d'un tableau existant

- NumPy permet de créer des vues ou des copies de tableaux existants.
- Lorsque vous effectuez des opérations sur un tableau, il est important de comprendre si vous travaillez sur une vue ou une copie du tableau d'origine.
- Pour créer une vue d'un tableau existant, vous pouvez utiliser la méthode `view()`.
- Pour créer une copie indépendante d'un tableau, vous pouvez utiliser la méthode `copy()`.

Exemple explicatif

```python
import numpy as np

# Création d'un tableau NumPy
original_array = np.array([1, 2, 3, 4, 5])

# Création d'une vue du tableau original
view_array = original_array.view()

# Création d'une copie indépendante du tableau original
copy_array = original_array.copy()

print("Original Array:", original_array)
print("View Array:", view_array)
'''
    References the original array,
    every change in original_array will be
    echoed into view_array and vice-versa.
'''
print("Copy Array:", copy_array)
```

Dimensions

- Les tableaux NumPy peuvent avoir différentes dimensions.
- La dimension d'un tableau NumPy fait référence au nombre des axes ou de dimensions du tableau.

L'attribut `ndim`

- L'attribut `ndim` de NumPy permet de connaître le nombre de dimensions d'un tableau.
- Cela retourne un entier représentant le nombre d'axes ou encore le nombre de dimensions du tableau.

```python
import numpy as np

# Creation of a NumPy array of 2 dimensions (matrix)
array_2d = np.array([[1, 2, 3], [4, 5, 6]])

# Getting the number of dimensions ndim
dimensions = array_2d.ndim

print("Number of dimensions:", dimensions)
```

L'attribut `size`

- L'attribut `size` de NumPy permet de connaître le nombre total d'éléments dans un tableau.
- Elle retourne un entier représentant la taille du tableau.

```python
import numpy as np

# Creation of a NumPy array of 3 dimensions (tensor)

array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]], [[9, 10], [11, 12]]])

# Getting the total size of the array
total_size = array_3d.size

print("Total size:", total_size)
```

L'attribut `shape`

- L'attribut `shape` de NumPy permet de connaître la forme ou les dimensions d'un tableau.
- Cela retourne un tuple représentant la forme du tableau.

```python
import numpy as np

# Création d'un tableau NumPy à deux dimensions
array_2d = np.array([[1, 2, 3], [4, 5, 6]])

# Obtenir la forme du tableau
shape = array_2d.shape

print("Shape:", shape)
```

L'attribut `dtype`

- Les tableaux NumPy peuvent contenir des éléments de différents types de données.
- Le type de données d'un tableau NumPy fait référence au type des éléments qu'il contient.
- L'attribut `dtype` de NumPy permet de connaître le type de données des éléments d'un tableau.
- Elle retourne un objet de type `dtype` représentant le type de données du tableau.

```python
import numpy as np

# Création d'un tableau NumPy d'entiers
array_int = np.array([1, 2, 3])

# Obtenir le type de données du tableau
data_type = array_int.dtype

print("Data type:", data_type)
```

La fonction `reshape()`

- Permet de changer la forme d'un tableau NumPy sans changer ses données.
- Prend en entrée les nouvelles dimensions souhaitées pour le tableau.
- Si le nombre total d'éléments du tableau reste le même, la fonction fonctionnera. Sinon, une erreur sera levée.

```python
import numpy as np

# Création d'un tableau 2D
arr = np.arange(1, 10).reshape((3, 3))
print(arr)
```

La fonction `squeeze()`

- Réduit les dimensions unitaires (de taille 1) d'un tableau NumPy.
- Retourne un nouveau tableau avec les dimensions unitaires supprimées.
- Utile pour réduire les dimensions superflues et simplifier les calculs.

```python
import numpy as np

# Création d'un tableau 3D avec une dimension unitaire
arr = np.ones((1, 3, 1))
print(arr.shape)  # Affiche (1, 3, 1)

# Suppression des dimensions unitaires
arr_squeezed = np.squeeze(arr)
print(arr_squeezed.shape)  # Affiche (3,)
```

La fonction `ravel()`

- Retourne un tableau aplati (unidimensionnel) contenant les mêmes données que le tableau d'origine.
- Ne crée pas une copie des données, mais retourne plutôt une vue des données d'origine.
- Peut être utilisé pour simplifier le traitement des données lorsque la forme du tableau n'est pas importante.

```python
import numpy as np

# Création d'un tableau 2D
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Utilisation de la fonction ravel()
arr_raveled = np.ravel(arr)
print(arr_raveled)
```

La fonction `flatten()`

- Similaire à la fonction `ravel()`, mais retourne une copie des données sous forme de tableau unidimensionnel.
- Crée une nouvelle copie des données plutôt que de retourner une vue.
- Utile lorsque vous voulez une copie indépendante des données originales.

```python
import numpy as np

# Création d'un tableau 2D
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Utilisation de la fonction flatten()
arr_flattened = arr.flatten()
print(arr_flattened)
```

La fonction `vstack()`

- Empile verticalement plusieurs tableaux NumPy en un seul.
- Prend en entrée une séquence de tableaux à empiler.
- Utile pour concaténer des tableaux le long de l'axe vertical (axe 0).

```python
import numpy as np

# Création de deux tableaux 1D
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

# Empilement vertical des tableaux
arr_vstacked = np.vstack((arr1, arr2))
print(arr_vstacked)
```

La fonction `hstack()`

- Empile horizontalement plusieurs tableaux NumPy en un seul.
- Prend en entrée une séquence de tableaux à empiler.
- Utile pour concaténer des tableaux le long de l'axe horizontal (axe 1).

```python
import numpy as np

# Création de deux tableaux 1D
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

# Empilement horizontal des tableaux
arr_hstacked = np.hstack((arr1, arr2))
print(arr_hstacked)
```

La fonction `concatenate()`

- Concatène plusieurs tableaux NumPy le long d'un axe spécifié.
- Prend en entrée une séquence de tableaux à concaténer ainsi que l'axe le long duquel effectuer la concaténation.
- Peut être utilisé pour concaténer des tableaux le long de n'importe quel axe.

```python
import numpy as np

# Création de deux tableaux 1D
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

# Concaténation des tableaux le long de l'axe 0
arr_concatenated = np.concatenate((arr1, arr2))
print(arr_concatenated)
```

Indexation

- Permet de naviguer dans un tableau NumPy à partir d'indices.
- Accède à des éléments individuels ou à des sous-tableaux en spécifiant les indices correspondants.

```python
import numpy as np

# Création d'un tableau NumPy
arr = np.array([1, 2, 3, 4, 5])

# Accès à un élément spécifique
print(arr[0])

# Accès à un sous-tableau
print(arr[1:3])
```

Découpage/Extraction

- Permet d'extraire une partie d'un tableau NumPy en spécifiant les indices de début, de fin et le pas.
- Utilise la syntaxe `[start:stop:step]` pour spécifier la plage d'indices à extraire.

```python
import numpy as np

# Création d'un tableau NumPy
arr = np.array([1, 2, 3, 4, 5])

# Extraction d'une partie du tableau
print(arr[1:4:2])
```

Introduction au Broadcasting

- La *diffusion* (ou *broadcasting*) est une fonctionnalité de NumPy permettant d'effectuer des opérations entre tableaux de formes différentes. NumPy **étend** les dimensions des tableaux plus petits pour les rendre compatibles avec les plus grands, sans créer de copies supplémentaires en mémoire.
- Le broadcasting est particulièrement utile pour éviter les boucles explicites, permettant des opérations plus rapides et plus lisibles (**vectorization**).
- Il est essentiel de comprendre les règles de compatibilité des formes pour éviter des erreurs lors des opérations.

Règles du Broadcasting

Voici comment NumPy applique le broadcasting entre deux tableaux :

- Le broadcasting fonctionne en comparant les dimensions de **droite à gauche**.
- Si les tableaux ***n'ont pas*** le même nombre de dimensions, NumPy **ajoute** des dimensions de taille 1 au début du plus petit tableau.
- Le broadcasting se base sur deux règles:
  - Soit deux dimensions d et $d^{'}$ sont égales *i.e.* : $d = d^{'}$;
  - Soit d **ou** $d^{'} = 1$;
  - Sinon le broadcasting n'est pas possible.

Logique mathématique du Broadcasting

La logique du broadcasting s'exprime ainsi :

- Soit deux tableaux de formes : $d_1 \times d_2 \times \ldots \times d_n$ et $d_1 \times d_2 \times \ldots \times d_m$, avec $n \geq m$.
- NumPy "étend" le deuxième tableau à $1 \times \ldots \times 1 \times d_1 \times d_2 \times \ldots \times d_m$, avec $n - m$ dimensions de 1 à gauche.
- Ensuite, NumPy applique le broadcasting dimension par dimension.
- Exemple :
  - Forme 1 : $4 \times 3 \times 2$
  - Forme 2 : 3×2
    Après broadcasting :
    - Forme 1 : $4 \times 3 \times 2 \Rightarrow 4 \times 3 \times 2$
    - Forme 2 : $3 \times 2 \Rightarrow 1 \times 3 \times 2$
    - Résultat: $4 \times 3 \times 2$

Exemple de Broadcasting élémentaire - Addition

Considérons une opération entre un tableau 2x3 et un vecteur de taille 3.

```python
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]])

B = np.array([10, 20, 30])

# Broadcasting entre un tableau de shape (2,3) et un vecteur de shape (3,) := (1,3)
C = A + B

print(C)
# Résultat :
# [[11 22 33]
#  [14 25 36]]
```

- NumPy ”étend” B de forme (3,):=(1,3) à (2, 3), puis applique le broadcasting.
- L’addition est effectuée element-wise.

Exemple de Broadcasting élémentaire - Multiplication

Considérons une opération entre un tableau 2x3 et un vecteur de taille 3.

```python
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]])

B = np.array([10, 20, 30])

# Broadcasting entre un tableau de shape (2,3) et un vecteur de shape (3,) := (1,3)
C = A * B

print(C)
# Résultat:
# [[ 10,  40,  90],
# [ 40, 100, 180]]
```

- NumPy ”étend” B de forme (3,) à (2, 3), puis applique le broadcasting.
- La multiplication est effectuée component-wise.

Remarque Importante

- **Remarque**. La *multiplication* dans le frame précédent est une forme de multiplication component-wise, à ne pas confondre avec le produit matriciel au sens usuel du terme !!

Broadcasting avec dimensions singletons

Un cas courant est l'ajout d'une nouvelle dimension à un tableau pour le rendre compatible pour le broadcasting.

```python
import numpy as np

# Tableau 3x2
A = np.array([[1],
              [2],
              [3]])

# Vecteur de taille 3
B = np.array([10, 20, 30])

# Broadcasting avec ajout de dimension
C = A * B

print(C)
# Résultat :
# [[10 20 30]
#  [20 40 60]
#  [30 60 90]]
```

Broadcasting avec des dimensions singletons - suite

- A est de forme (3, 1) et B est de forme (3,) := (1,3).
- NumPy étend B pour qu'il ait la forme (2, 3), permettant ainsi la multiplication élément par élément.

Broadcasting avec des scalaires

Le broadcasting peut également être appliqué lors d'opérations avec des scalaires, ce qui est une forme simple mais puissante de diffusion.

```python
import numpy as np

# Tableau 3x3
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Scalaire
B = 10

# Broadcasting d'un scalaire sur un tableau
C = A + B

print(C)
# Résultat :
# [[11 12 13]
#  [14 15 16]
#  [17 18 19]]
```

Broadcasting avec des scalaires - suite

- Ici, le scalaire B est étendu pour avoir la même forme que A (3, 3).
- Le broadcasting s'applique à chaque élément de A, ajoutant 10 à chaque valeur.

Broadcasting sur plusieurs axes

Le broadcasting peut également se produire sur plusieurs axes simultanément.

```python
import numpy as np

# Tableau 2x1x3
A = np.array([[[1, 2, 3]],
              [[4, 5, 6]]])

# Tableau 1x2x1
B = np.array([[[10], [20]]])

# Broadcasting sur plusieurs axes
C = A + B

print(C)
# Résultat :
# [[[11 12 13]
#   [21 22 23]]
#
#  [[14 15 16]
#   [24 25 26]]]
```

Broadcasting sur plusieurs axes - suite

- A est de forme (2, 1, 3) et B est de forme (1, 2, 1).
- NumPy étend A et B pour qu'ils aient la forme (2, 2, 3), puis applique l'addition.
- Le résultat est un tableau de forme (2, 2, 3) où les éléments de B sont ajoutés aux éléments correspondants de A.

Broadcasting avec des dimensions supplémentaires

- Le broadcasting se produit lorsque les tableaux ont des dimensions supplémentaires qui peuvent être étendues pour correspondre.
- Par exemple, lorsqu'un tableau 4D est combiné avec un tableau 3D, NumPy applique le broadcasting sur les dimensions qui diffèrent.

Exemple

```python
import numpy as np

# Tableau 2x1x3x1 (4D)
A = np.array([[[[1], [2], [3]]], [[[4], [5], [6]]]])

# Tableau 1x3x5 (3D)
B = np.array([[[10, 20, 30, 40, 50],
               [60, 70, 80, 90, 100],
               [110, 120, 130, 140, 150]]])

# Broadcasting avec des dimensions supplémentaires
C = A + B

print(C)
# Résultat :
# [[[[ 11  21  31  41  51]
#    [ 12  22  32  42  52]
#    [ 13  23  33  43  53]]]
#
#  [[[ 14  24  34  44  54]
#    [ 15  25  35  45  55]
#    [ 16  26  36  46  56]]]]
```

Broadcasting avec des dimensions supplémentaires - suite

- A est de forme (2, 1, 3, 1) et B est de forme (1, 3, 5).
- NumPy étend A à (2, 1, 3, 5) et B à (2, 1, 3, 5) pour qu'ils aient la même forme, puis applique l'addition.
- Le résultat est un tableau de forme (2, 1, 3, 5) où les éléments de B sont ajoutés aux éléments correspondants de A.

Erreurs de Broadcasting

Le broadcasting ne fonctionne que si les dimensions des tableaux sont compatibles. Dans certains cas, une opération peut échouer si les règles ne sont pas respectées.

- Les dimensions doivent être égales ou l'une d'entre elles doit être 1.
- Si les formes ne sont pas compatibles, NumPy lève une erreur `ValueError`.

Exemple d'erreur de Broadcasting

Voyons un cas où le broadcasting échoue en raison d'incompatibilités de forme.

```python
import numpy as np

# Tableau 2x3
A = np.array([[1, 2, 3],
              [4, 5, 6]])

# Tableau 2x2 (incompatible pour broadcasting avec A)
B = np.array([[10, 20],
              [30, 40]])

try:
    C = A + B
except ValueError as e:
    print(f"Erreur de broadcasting: {e}")
```

- A est de forme (2, 3) et B est de forme (2, 2).
- Comme les dimensions ne sont pas compatibles, une erreur de `ValueError` est levée.

Sélection/Masquage

- Permet de vérifier une condition pour chaque élément d'un tableau NumPy.
- Crée un masque booléen indiquant quels éléments du tableau satisfont la condition spécifiée.

```python
import numpy as np

# Création d'un tableau NumPy
arr = np.array([1, 2, 3, 4, 5])

# Masquage des éléments supérieurs à 3
mask = arr > 3
print(mask)
```

Indexation booléenne

- Permet de naviguer conditionnellement dans un tableau NumPy en sélectionnant les éléments répondant à une condition donnée.
- Utilise un masque booléen pour sélectionner les éléments qui satisfont la condition.

```python
import numpy as np

# Création d'un tableau NumPy
arr = np.array([1, 2, 3, 4, 5])

# Sélection des éléments supérieurs à 3
selected = arr[arr > 3]
print(selected)
```

** Pandas **

Introduction à Pandas - Présentation

- Pandas est la bibliothèque Python incontournable pour l'analyse de données tabulaires
- Créée par Wes McKinney en 2008, devenue standard de facto en data science
- Construite sur NumPy, offre des structures de données performantes et intuitives
- Idéale pour : nettoyage, transformation, agrégation et analyse exploratoire

```python
import pandas as pd
import numpy as np

# Check version
print(f"Pandas version: {pd.__version__}")
# Output: Pandas version: 3.0.1
```

Introduction à Pandas - Structures principales

- **Series** : tableau unidimensionnel étiqueté (équivalent d'une colonne)
- **DataFrame** : tableau bidimensionnel étiqueté (équivalent d'une table SQL)
- Les deux structures supportent l'indexation flexible et les données hétérogènes
- Optimisées pour les performances avec des opérations vectorisées

```python
# Simple Series creation
series_example = pd.Series([10, 20, 30, 40], name='values')

# Simple DataFrame creation
df_example = pd.DataFrame({
    'column_a': [1, 2, 3],
    'column_b': ['x', 'y', 'z']
})
```

Les Séries - Création et propriétés

- Une Series est un array 1D avec un index explicite
- Peut contenir n'importe quel type de données (int, float, str, object)
- L'index par défaut est numérique (0, 1, 2, ...) mais peut être personnalisé
- Supporte les opérations vectorisées et l'alignement automatique par index

```python
# Series with default integer index
series_default = pd.Series([100, 200, 300, 400])

# Series with custom index
series_custom = pd.Series(
    data=[100, 200, 300, 400],
    index=['a', 'b', 'c', 'd'],
    name='sales'
)

# Series from dictionary
series_dict = pd.Series({'Jan': 1500, 'Feb': 1800, 'Mar': 2100})
```

Les Séries - Accès et manipulation

- Accès par position (iloc) ou par label (loc)
- Slicing et filtrage conditionnels disponibles
- Méthodes d'agrégation : sum(), mean(), median(), std(), etc.
- Opérations element-wise entre Series alignées par index

```python
temperature = pd.Series([22.5, 23.1, 21.8, 24.2], index=['Mon', 'Tue', 'Wed', 'Thu'])

# Access by label
monday_temp = temperature['Mon']  # 22.5

# Access by position
first_temp = temperature.iloc[0]  # 22.5

# Boolean filtering
hot_days = temperature[temperature > 23.0]

# Statistical operations
avg_temp = temperature.mean()
std_temp = temperature.std()
```

Les Séries - Opérations avancées

- Application de fonctions : apply(), map(), replace()
- Gestion des valeurs manquantes : isna(), fillna(), dropna()
- Tri par valeur ou par index : sort_values(), sort_index()
- Conversion de type : astype(), to_numpy(), to_dict()

```python
scores = pd.Series([85, 92, np.nan, 78, 95], index=['Alice', 'Bob', 'Carol', 'Dave', 'Eve'])

# Apply function
normalized = scores.apply(lambda x: x / 100)

# Fill missing values
scores_filled = scores.fillna(scores.mean())

# Sort by values descending
scores_sorted = scores.sort_values(ascending=False)

# Convert to dictionary
scores_dict = scores.to_dict()
```

Les DataFrames - Création et structure

- DataFrame = dictionnaire de Series partageant le même index
- Peut être créé depuis : dict, list of dicts, numpy array, autre DataFrame
- Possède deux index : un pour les lignes, un pour les colonnes
- Chaque colonne peut avoir un type de données différent

```python
# From dictionary of lists
df_from_dict = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Carol'],
    'age': [25, 30, 27],
    'salary': [50000, 60000, 55000]
})

# From list of dictionaries
data_list = [
    {'name': 'Alice', 'age': 25, 'salary': 50000},
    {'name': 'Bob', 'age': 30, 'salary': 60000},
    {'name': 'Carol', 'age': 27, 'salary': 55000}
]
df_from_list = pd.DataFrame(data_list)
```

Les DataFrames - Propriétés et inspection

- Méthodes d'inspection : head(), tail(), info(), describe()
- Attributs utiles : shape, columns, index, dtypes
- Statistiques descriptives automatiques pour colonnes numériques
- Aperçu rapide de la structure et qualité des données

```python
df = pd.DataFrame({
    'product': ['A', 'B', 'C', 'D'],
    'price': [19.99, 29.99, 15.49, 39.99],
    'quantity': [100, 50, 200, 75]
})

# Inspection methods
print(df.head(2))  # First 2 rows
print(df.info())   # Column types and non-null counts
print(df.describe())  # Statistical summary

# Properties
rows, cols = df.shape
column_names = df.columns.tolist()
data_types = df.dtypes
```

Les DataFrames - Ajout et suppression

- Ajout de colonnes : assignation directe ou méthode assign()
- Ajout de lignes : append() (deprecated), concat(), loc[]
- Suppression : drop() pour lignes ou colonnes
- Les opérations peuvent être inplace ou retourner une copie

```python
df = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})

# Add column
df['c'] = df['a'] + df['b']

# Add column with assign (functional style)
df = df.assign(d=lambda x: x['a'] * 2)

# Drop column
df_dropped = df.drop(columns=['d'])

# Drop row by index
df_dropped_row = df.drop(index=0)

# Drop inplace
df.drop(columns=['c'], inplace=True)
```

Lecture de données - CSV et texte

- read_csv() : format le plus courant, très flexible
- Paramètres importants : sep, header, names, index_col, dtype
- Gestion automatique de la compression (gzip, bz2, zip)
- Parsing intelligent des dates avec parse_dates

```python
# Basic CSV reading
df_csv = pd.read_csv('data.csv')

# Advanced CSV reading
df_advanced = pd.read_csv(
    'data.csv',
    sep=';',              # Custom separator
    header=0,             # Row number for column names
    index_col='id',       # Set index column
    parse_dates=['date'], # Parse date columns
    dtype={'category': 'category'},  # Specify dtypes
    na_values=['NA', 'null', '-']    # Custom NA values
)

# Read from URL
url = 'https://example.com/data.csv'
df_url = pd.read_csv(url)
```

Lecture de données - Excel et autres formats

- read_excel() : lecture de fichiers .xlsx, .xls
- read_json() : données JSON (orientations multiples)
- read_sql() : requêtes SQL directes
- read_parquet(), read_feather() : formats optimisés pour big data

```python
# Excel reading
df_excel = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# Multiple sheets
sheets_dict = pd.read_excel('data.xlsx', sheet_name=None)

# JSON reading
df_json = pd.read_json('data.json', orient='records')

# SQL reading (requires sqlalchemy)
from sqlalchemy import create_engine
engine = create_engine('sqlite:///database.db')
df_sql = pd.read_sql('SELECT * FROM table_name', con=engine)

# Parquet (efficient columnar format)
df_parquet = pd.read_parquet('data.parquet')
```

Export de données - Formats multiples

- to_csv() : export vers CSV, options de formatage nombreuses
- to_excel() : export vers Excel (nécessite openpyxl ou xlsxwriter)
- to_json(), to_sql(), to_parquet() : autres formats
- Paramètre index=False souvent utile pour éviter l'export de l'index

```python
df = pd.DataFrame({'col1': [1, 2, 3], 'col2': ['a', 'b', 'c']})

# CSV export
df.to_csv('output.csv', index=False, sep=';', encoding='utf-8')

# Excel export
df.to_excel('output.xlsx', sheet_name='Data', index=False)

# JSON export
df.to_json('output.json', orient='records', indent=2)

# SQL export
df.to_sql('table_name', con=engine, if_exists='replace', index=False)

# Parquet export (compressed, efficient)
df.to_parquet('output.parquet', compression='gzip')
```

Sélection - Colonnes et lignes basiques

- Accès aux colonnes : df['col'] ou df.col (notation point)
- Sélection multiple : df[['col1', 'col2']]
- Slicing de lignes : df[0:5] (par position uniquement)
- Filtrage booléen : df[condition]

```python
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Carol', 'Dave'],
    'age': [25, 30, 27, 35],
    'city': ['Paris', 'Lyon', 'Paris', 'Marseille']
})

# Single column (returns Series)
ages = df['age']
# Multiple columns (returns DataFrame)
subset = df[['name', 'city']]
# Row slicing
first_two = df[0:2]
# Boolean filtering
parisians = df[df['city'] == 'Paris']
adults = df[df['age'] >= 30]
```

Sélection - loc et iloc

- **loc[]** : sélection par labels (index et colonnes)
- **iloc[]** : sélection par positions entières
- Syntaxe : df.loc[lignes, colonnes] ou df.iloc[lignes, colonnes]
- Supporte le slicing, les listes, et les booléens

```
df = pd.DataFrame(
    {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]},
    index=['x', 'y', 'z']
)

# loc: by label
value = df.loc['x', 'A']  # 1
row = df.loc['x']         # First row as Series
subset = df.loc['x':'y', ['A', 'C']]

# iloc: by position
value = df.iloc[0, 0]     # 1
row = df.iloc[0]          # First row
subset = df.iloc[0:2, [0, 2]]

# Boolean with loc
df.loc[df['A'] > 1, 'B'] = 999  # Conditional assignment
```

Sélection - Techniques avancées

- query() : sélection avec syntaxe SQL-like
- at[] et iat[] : accès rapide à une valeur scalaire
- filter() : sélection de colonnes par pattern
- Chaînage de conditions multiples avec & et |

```python
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Carol'],
    'age': [25, 30, 27],
    'salary': [50000, 60000, 55000]
})

# Query method (expression as string)
young = df.query('age < 28')
high_earners = df.query('salary > 52000 and age < 30')
# Fast scalar access
value = df.at[0, 'name']  # 'Alice'
value = df.iat[0, 0]      # 'Alice' (by position)
# Filter columns by pattern
name_cols = df.filter(like='name')
regex_cols = df.filter(regex='^[as]')  # Columns starting with 'a' or 's'
```

Opérations arithmétiques et statistiques

- Opérations vectorisées sur colonnes numériques
- Broadcasting automatique pour scalaires
- Méthodes d'agrégation : sum(), mean(), median(), std(), min(), max()
- Opérations element-wise entre DataFrames alignés

```python
df = pd.DataFrame({
    'price': [100, 200, 150, 180],
    'quantity': [2, 1, 3, 2]
})

# Arithmetic operations
df['total'] = df['price'] * df['quantity']
df['discounted_price'] = df['price'] * 0.9

# Statistical aggregations
avg_price = df['price'].mean()
total_sales = df['total'].sum()
price_range = df['price'].max() - df['price'].min()

# Column-wise operations
stats = df[['price', 'quantity']].agg(['mean', 'std', 'min', 'max'])
```

Tri et classement

- sort_values() : tri par valeurs d'une ou plusieurs colonnes
- sort_index() : tri par index
- rank() : calcul de rangs
- nlargest() et nsmallest() : top N valeurs

```python
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Carol', 'Dave'],
    'score': [85, 92, 78, 92],
    'age': [25, 30, 27, 35]
})

# Sort by single column
sorted_by_score = df.sort_values('score', ascending=False)
# Sort by multiple columns
sorted_multi = df.sort_values(['score', 'age'], ascending=[False, True])
# Rank calculation
df['rank'] = df['score'].rank(ascending=False, method='min')
# Top N
top_3 = df.nlargest(3, 'score')
bottom_2 = df.nsmallest(2, 'age')
```

Agrégation et groupement

- groupby() : regroupement par une ou plusieurs colonnes
- Fonctions d'agrégation : sum(), mean(), count(), etc.
- agg() : application de plusieurs fonctions simultanément
- transform() : opérations qui conservent la forme originale

```python
df = pd.DataFrame({
    'category': ['A', 'B', 'A', 'B', 'A'],
    'value': [10, 20, 15, 25, 12],
    'quantity': [1, 2, 1, 3, 2]
})

grouped = df.groupby('category')['value'].mean() # Basic groupby
# Multiple aggregations
agg_result = df.groupby('category').agg({
    'value': ['sum', 'mean', 'count'],
    'quantity': 'sum'
})
# Transform (returns same shape)
df['value_normalized'] = df.groupby('category')['value'].transform(
    lambda x: (x - x.mean()) / x.std()
)
```

Détection des valeurs manquantes

- Pandas représente les valeurs manquantes par NaN (NumPy) ou None
- isna() / isnull() : détection des valeurs manquantes
- notna() / notnull() : détection des valeurs présentes
- info() montre le nombre de non-null par colonne

```
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [5, np.nan, np.nan, 8],
    'C': [9, 10, 11, 12]
})

# Check for missing values
missing_mask = df.isna()
# Count missing values per column
missing_count = df.isna().sum()
# Percentage of missing values
missing_pct = (df.isna().sum() / len(df)) * 100
# Rows with any missing value
rows_with_na = df[df.isna().any(axis=1)]
# Columns with any missing value
cols_with_na = df.columns[df.isna().any()].tolist()
```

Suppression des valeurs manquantes

- dropna() : suppression des lignes ou colonnes avec NaN
- Paramètres : axis (0=lignes, 1=colonnes), how ('any' ou 'all')
- thresh : nombre minimum de valeurs non-null requises
- subset : limiter la vérification à certaines colonnes

```python
df = pd.DataFrame({
    'A': [1, np.nan, 3, 4],
    'B': [5, 6, np.nan, 8],
    'C': [9, 10, 11, 12]
})

# Drop rows with any NaN
df_dropped_rows = df.dropna()
# Drop columns with any NaN
df_dropped_cols = df.dropna(axis=1)
# Drop rows where all values are NaN
df_all = df.dropna(how='all')
# Keep rows with at least 2 non-NaN values
df_thresh = df.dropna(thresh=2)
# Drop based on specific columns
df_subset = df.dropna(subset=['A', 'B'])
```

Remplissage des valeurs manquantes

- fillna() : remplacement des NaN par une valeur
- Méthodes : valeur constante, forward fill (ffill), backward fill (bfill)
- interpolate() : interpolation linéaire ou polynomiale
- Remplissage par statistiques : mean(), median(), mode()

```
df = pd.DataFrame({
    'A': [1, np.nan, 3, np.nan, 5],
    'B': [10, 20, np.nan, 40, 50]
})

# Fill with constant value
df_filled = df.fillna(0)
# Forward fill (propagate last valid value)
df_ffill = df.fillna(method='ffill')
# Fill with column mean
df['A'] = df['A'].fillna(df['A'].mean())
# Fill with different values per column
df_filled_dict = df.fillna({'A': 0, 'B': df['B'].median()})
# Linear interpolation
df_interpolated = df.interpolate(method='linear')
```

Fusion et jointures - merge()

- merge() : jointures SQL-like entre DataFrames
- Types : inner, left, right, outer
- Jointure sur colonnes (on) ou sur index (left_index, right_index)
- Gestion des suffixes pour colonnes dupliquées

```
df_customers = pd.DataFrame({
    'customer_id': [1, 2, 3],
    'name': ['Alice', 'Bob', 'Carol']
})
df_orders = pd.DataFrame({
    'order_id': [101, 102, 103],
    'customer_id': [1, 2, 4],
    'amount': [250, 150, 300]
})

# Inner join (default)
merged_inner = pd.merge(df_customers, df_orders, on='customer_id')
# Left join (keep all customers)
merged_left = pd.merge(df_customers, df_orders, on='customer_id', how='left')
# Outer join (keep all records)
merged_outer = pd.merge(df_customers, df_orders, on='customer_id', how='outer')
```

Concaténation et combinaison

- concat() : empilement vertical ou horizontal de DataFrames
- append() : ajout de lignes (deprecated, utiliser concat)
- join() : jointure sur index
- combine_first() : remplissage prioritaire

```python
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})

# Vertical concatenation (stack rows)
df_concat_vert = pd.concat([df1, df2], axis=0, ignore_index=True)

# Horizontal concatenation (stack columns)
df_concat_horiz = pd.concat([df1, df2], axis=1)

# Join on index
df_joined = df1.join(df2, lsuffix='_left', rsuffix='_right')

# Combine with priority
df_base = pd.DataFrame({'A': [1, np.nan], 'B': [3, 4]})
df_filler = pd.DataFrame({'A': [10, 20], 'B': [30, 40]})
df_combined = df_base.combine_first(df_filler)
```

Pivot et restructuration

- pivot() : transformation lignes vers colonnes (format wide)
- pivot_table() : pivot avec agrégation
- melt() : transformation colonnes vers lignes (format long)
- stack() / unstack() : pivoting sur index multi-niveaux

```python
df_long = pd.DataFrame({
    'date': ['2024-01-01', '2024-01-01', '2024-01-02', '2024-01-02'],
    'city': ['Paris', 'Lyon', 'Paris', 'Lyon'],
    'temperature': [10, 12, 11, 13]
})
# Pivot to wide format
df_wide = df_long.pivot(index='date', columns='city', values='temperature')

# Pivot table with aggregation
df_pivot = df_long.pivot_table(
    index='date',
    columns='city',
    values='temperature',
    aggfunc='mean'
)
# Melt to long format
df_melted = df_wide.reset_index().melt(id_vars='date', var_name='city', value_name='temp')
```

Apply, map et transformation de données

- apply() : application de fonction sur lignes ou colonnes
- map() : mapping élément par élément (Series uniquement)
- applymap() : application élément par élément sur DataFrame (deprecated)
- replace() : remplacement de valeurs spécifiques

```python
df = pd.DataFrame({
    'name': ['alice', 'bob', 'carol'],
    'score': [85, 92, 78]
})
df['name_upper'] = df['name'].apply(str.upper) # Apply function to column
# Apply function to rows (axis=1)
df['grade'] = df.apply(
    lambda row: 'A' if row['score'] >= 90 else 'B' if row['score'] >= 80 else 'C',
    axis=1
)
grade_map = {85: 'B', 92: 'A', 78: 'C'}
df['grade_mapped'] = df['score'].map(grade_map) # Map with dictionary
df['name'] = df['name'].replace('alice', 'Alice') # Replace values
```

Création d'index hiérarchiques

- MultiIndex : indexation multi-niveaux pour lignes ou colonnes
- Création via from_tuples(), from_arrays(), from_product()
- set_index() : promotion de colonnes vers l'index
- reset_index() : conversion d'index vers colonnes

```python
# Create MultiIndex from tuples
index = pd.MultiIndex.from_tuples([
    ('A', 'x'), ('A', 'y'), ('B', 'x'), ('B', 'y')
], names=['category', 'subcategory'])
df = pd.DataFrame({'value': [10, 20, 30, 40]}, index=index)
# Create MultiIndex from arrays
arrays = [
    ['A', 'A', 'B', 'B'],
    ['x', 'y', 'x', 'y']
]
index = pd.MultiIndex.from_arrays(arrays, names=['cat', 'subcat'])
# Set MultiIndex from columns
df_flat = pd.DataFrame({
    'cat': ['A', 'A', 'B'], 'subcat': ['x', 'y', 'x'], 'val': [1, 2, 3]
})
df_multi = df_flat.set_index(['cat', 'subcat'])
```

GroupBy avec MultiIndex

- groupby() avec plusieurs colonnes crée automatiquement un MultiIndex
- Agrégations multi-niveaux pour analyses complexes
- as_index=False pour éviter le MultiIndex
- Combinaison avec pivot_table() pour analyses croisées

```
df = pd.DataFrame({
    'region': ['North', 'North', 'South', 'South'],
    'product': ['A', 'B', 'A', 'B'],
    'sales': [100, 150, 120, 180],
    'quantity': [10, 15, 12, 18]
})
# GroupBy creating MultiIndex
grouped = df.groupby(['region', 'product']).agg({'sales': 'sum', 'quantity': 'sum'})
# Result has MultiIndex (region, product)
grouped_flat = df.groupby(['region', 'product'], as_index=False).sum() # Avoid MultiIndex
pivot = df.pivot_table(index='region', columns='product', values='sales', aggfunc='sum') # Pivot table with
    MultiIndex
```

Sélection dans les MultiIndex

- Sélection par tuple : df.loc[('level1', 'level2')]
- Slicing multi-niveaux avec pd.IndexSlice
- xs() : cross-section pour sélection à un niveau spécifique
- swaplevel() : échange de niveaux d'index

```python
index = pd.MultiIndex.from_product([['A', 'B'], ['x', 'y']], names=['L1', 'L2'])
df = pd.DataFrame({'value': [10, 20, 30, 40]}, index=index)

# Select by tuple
value_ax = df.loc[('A', 'x')]
# Select all 'x' in level 2
all_x = df.loc[(slice(None), 'x'), :]
# Using IndexSlice for complex selection
idx = pd.IndexSlice
subset = df.loc[idx['A', :], :]
# Cross-section (all level L1='A')
df_a = df.xs('A', level='L1')
# Swap levels
df_swapped = df.swaplevel('L1', 'L2')
```

Opérations sur MultiIndex

- stack() : pivoting vers MultiIndex (colonnes -> index)
- unstack() : pivoting depuis MultiIndex (index -> colonnes)
- reset_index() : aplatissement du MultiIndex
- droplevel() : suppression d'un niveau d'index

```python
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])

# Stack: columns to index
df_stacked = df.stack()
# Result: MultiIndex (original_index, column_name)
# Unstack: index to columns
df_unstacked = df_stacked.unstack()
# Reset MultiIndex to columns
df_multi = df.stack().to_frame('value')
df_flat = df_multi.reset_index()
# Drop level from MultiIndex
df_multi.index = df_multi.index.droplevel(1)
```

Agrégations sur MultiIndex

- Agrégation par niveau avec level parameter
- sum(), mean() etc. supportent l'agrégation par niveau
- Utilisation avec groupby() pour analyses hiérarchiques
- Combinaison avec unstack() pour tableaux croisés

```python
index = pd.MultiIndex.from_tuples([
    ('2024', 'Q1'), ('2024', 'Q2'), ('2025', 'Q1'), ('2025', 'Q2')
], names=['year', 'quarter'])

df = pd.DataFrame({'revenue': [100, 150, 120, 180]}, index=index)

# Sum by first level (year)
yearly_total = df.sum(level='year')
# Sum by second level (quarter)
quarterly_total = df.sum(level='quarter')
# Unstack for pivot view
pivot_view = df.unstack(level='quarter')
# GroupBy on one level
by_year = df.groupby(level='year').mean()
```

Opérations de base sur les chaînes

- Accesseur .str pour méthodes de chaînes vectorisées
- Méthodes courantes : lower(), upper(), strip(), replace()
- Découpage : split(), slice()
- Vérifications : contains(), startswith(), endswith()

```python
df = pd.DataFrame({
    'name': ['  Alice Smith  ', 'bob jones', 'CAROL BROWN'],
    'email': ['alice@example.com', 'bob@test.com', 'carol@demo.com']
})

# Case transformation
df['name_lower'] = df['name'].str.lower()
df['name_upper'] = df['name'].str.upper()
# Strip whitespace
df['name_clean'] = df['name'].str.strip()
# String replacement
df['email_masked'] = df['email'].str.replace('@', '[at]')
# Split strings
df['domain'] = df['email'].str.split('@').str[1]
# Check contains
has_alice = df['name'].str.contains('Alice', case=False)
```

Extraction et recherche de patterns

- extract() : extraction via regex avec groupes nommés
- findall() : recherche de toutes les occurrences
- match() : correspondance au début de la chaîne
- len() : longueur des chaînes

```python
df = pd.DataFrame({
    'text': ['Order #12345', 'Invoice #67890', 'Receipt #11111'],
    'email': ['user123@domain.com', 'admin@site.org', 'test99@mail.net']
})

# Extract pattern with named groups
pattern = r'#(?P<number>\d+)'
df['order_num'] = df['text'].str.extract(pattern)
# Extract email components
email_pattern = r'(?P<user>\w+)@(?P<domain>[\w.]+)'
df_extracted = df['email'].str.extract(email_pattern)
# Find all digits
df['all_digits'] = df['text'].str.findall(r'\d+')
# String length
df['text_length'] = df['text'].str.len()
```

Formatage et concaténation

- cat() : concaténation de chaînes avec séparateur
- zfill() : padding avec zéros
- pad() : padding gauche/droite
- format() : formatage avec templates

```python
df = pd.DataFrame({
    'first_name': ['Alice', 'Bob', 'Carol'],
    'last_name': ['Smith', 'Jones', 'Brown'],
    'id': [1, 2, 3]
})

# Concatenate columns
df['full_name'] = df['first_name'].str.cat(df['last_name'], sep=' ')
# Alternative: simple concatenation
df['full_name_alt'] = df['first_name'] + ' ' + df['last_name']
# Zero-fill ID
df['id_padded'] = df['id'].astype(str).str.zfill(5)
# Left pad
df['id_left_pad'] = df['id'].astype(str).str.pad(width=5, side='left', fillchar='0')
# Format template
df['formatted'] = df.apply(lambda x: f"User: {x['full_name']} (ID: {x['id']:03d})", axis=1)
```

Regex basiques avec Pandas

- contains() avec regex=True : recherche de patterns
- replace() avec regex=True : remplacement de patterns
- extract() et extractall() : extraction de groupes
- Flags regex : case, flags=re.IGNORECASE

```python
import re

df = pd.DataFrame({
    'text': ['Product-A-123', 'Item-B-456', 'Service-C-789', 'Product-D-012']
})

# Contains with regex
has_product = df['text'].str.contains(r'^Product', regex=True)
# Extract pattern
pattern = r'([A-Z])-(\d+)'
df_extracted = df['text'].str.extract(pattern)
df_extracted.columns = ['letter', 'number']
# Replace with regex
df['cleaned'] = df['text'].str.replace(r'\d+', 'XXX', regex=True)
# Case insensitive search
has_product_ci = df['text'].str.contains('product', case=False, regex=True)
```

Patterns regex avancés

- Groupes de capture : (?P<name>pattern)
- Lookahead et lookbehind : (?=...), (?<=...)
- Quantificateurs : *, +, ?, {n,m}
- Caractères spéciaux : \d, \w, \s

```
df = pd.DataFrame({
    'log': ['[ERROR] Connection failed at 14:32',
            '[INFO] System started at 09:15',
            '[WARNING] Memory low at 16:45']
})

# Extract with named groups
pattern = r'\[(?P<level>\w+)\] (?P<message>.*?) at (?P<time>\d{2}:\d{2})'
df_parsed = df['log'].str.extract(pattern)
# Extract email parts
emails = pd.Series(['user@domain.com', 'admin@site.co.uk'])
email_pattern = r'(?P<user>[\w.]+)@(?P<domain>[\w.]+)\.(?P<tld>\w+)'
email_parts = emails.str.extract(email_pattern)
# Find all numbers
df['all_numbers'] = df['log'].str.findall(r'\d+')
```

Validation et nettoyage avec regex

- Validation de formats : email, téléphone, codes postaux
- Nettoyage de données : suppression de caractères spéciaux
- Normalisation : standardisation de formats
- Extraction d'informations structurées

```python
df = pd.DataFrame({
    'email': ['valid@email.com', 'invalid.email', 'another@test.co.uk'],
    'phone': ['+33-1-23-45-67-89', '01.23.45.67.89', '(01) 23 45 67 89']
})

# Validate email format
email_pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
df['email_valid'] = df['email'].str.match(email_pattern)
# Normalize phone numbers (remove non-digits)
df['phone_normalized'] = df['phone'].str.replace(r'[^\d]', '', regex=True)
# Extract area code
df['area_code'] = df['phone'].str.extract(r'^[\+\(]?(\d{2})')
# Clean: remove special characters
dirty = pd.Series(['Hello@World!', 'Test#123', 'Clean_Text'])
cleaned = dirty.str.replace(r'[^\w\s]', '', regex=True)
```

Création et parsing de dates

- to_datetime() : conversion vers type datetime
- date_range() : génération de séquences de dates
- Formats automatiques et personnalisés
- Gestion des fuseaux horaires (timezone-aware)

```python
import pandas as pd

# Parse dates from strings
dates_str = ['2024-01-15', '2024-02-20', '2024-03-25']
dates = pd.to_datetime(dates_str)
# Custom format parsing
dates_custom = pd.to_datetime(['15/01/2024', '20/02/2024'], format='%d/%m/%Y')
# Generate date range
date_range = pd.date_range(start='2024-01-01', end='2024-12-31', freq='D')
# Business days only
business_days = pd.bdate_range(start='2024-01-01', end='2024-01-31')
# With timezone
dates_tz = pd.date_range('2024-01-01', periods=5, freq='D', tz='Europe/Paris')
```

Extraction de composants temporels

- Accesseur .dt pour attributs temporels
- Extraction : year, month, day, hour, minute, second
- Propriétés calculées : dayofweek, dayofyear, quarter
- Noms : day_name(), month_name()

```python
df = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=10, freq='D'),
    'value': range(10)
})
# Extract components
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['dayofweek'] = df['date'].dt.dayofweek  # 0=Monday, 6=Sunday
df['quarter'] = df['date'].dt.quarter
# Named components
df['day_name'] = df['date'].dt.day_name()
df['month_name'] = df['date'].dt.month_name()
# Is weekend?
df['is_weekend'] = df['date'].dt.dayofweek.isin([5, 6])
```

Resampling et agrégation temporelle

- resample() : ré-échantillonnage temporel avec agrégation
- Fréquences : 'D' (jour), 'W' (semaine), 'M' (mois), 'Q' (trimestre), 'Y' (année)
- Méthodes d'agrégation : sum(), mean(), first(), last()
- rolling() : fenêtres glissantes

```python
df = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=100, freq='D'),
    'sales': np.random.randint(100, 500, 100)
})
df = df.set_index('date')

# Daily to weekly aggregation
weekly_sales = df.resample('W').sum()
# Daily to monthly average
monthly_avg = df.resample('M').mean()
# Rolling window (7-day moving average)
df['sales_ma7'] = df['sales'].rolling(window=7).mean()
# Rolling sum
df['sales_sum7'] = df['sales'].rolling(window=7).sum()
```

Opérations temporelles avancées

- Décalages temporels : shift(), diff()
- Timedelta : calcul de durées
- Calculs entre dates : différences, additions
- Filtrage par plage de dates

```python
df = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=10, freq='D'),
    'price': [100, 102, 105, 103, 107, 110, 108, 112, 115, 113]
})
df = df.set_index('date')
# Shift values (lag)
df['price_prev'] = df['price'].shift(1)
# Calculate difference
df['price_change'] = df['price'].diff()
# Percentage change
df['price_pct_change'] = df['price'].pct_change()
# Timedelta calculation
df['days_since_start'] = df.index - df.index[0]
# Filter by date range
mask = (df.index >= '2024-01-05') & (df.index <= '2024-01-08')
df_filtered = df.loc[mask]
```

Chunking et traitement par morceaux

- read_csv() avec chunksize : lecture par blocs
- Traitement itératif pour économiser la mémoire
- Agrégation progressive des résultats
- Adaptation pour fichiers de plusieurs Go

```python
# Read large file in chunks
chunk_size = 10000
chunks = []
for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
    # Process each chunk
    chunk_processed = chunk[chunk['value'] > 100]
    chunks.append(chunk_processed)

# Combine all chunks
df_final = pd.concat(chunks, ignore_index=True)

# Alternative: aggregate on the fly
total = 0
for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
    total += chunk['amount'].sum()
print(f"Total amount: {total}")
```

Optimisation de la mémoire

- Typage optimal : category pour chaînes répétitives
- Downcasting : int64 -> int32 ou int16 si possible
- memory_usage() : analyse de la consommation mémoire
- Suppression de colonnes inutiles dès la lecture

```python
# Read only needed columns
df = pd.read_csv('data.csv', usecols=['id', 'name', 'value'])
# Optimize data types
df['id'] = df['id'].astype('int32')
df['name'] = df['name'].astype('category')
# Check memory usage
print(df.memory_usage(deep=True))
# Downcast numeric types
df['value'] = pd.to_numeric(df['value'], downcast='float')
# Before and after comparison
def optimize_df(df):
    for col in df.select_dtypes(include=['int64']).columns:
        df[col] = pd.to_numeric(df[col], downcast='integer')
    for col in df.select_dtypes(include=['float64']).columns:
        df[col] = pd.to_numeric(df[col], downcast='float')
    return df
```

Formats alternatifs : Parquet et Feather

- Parquet : format columnar compressé, idéal pour l'analytique
- Feather : format binaire rapide pour sérialisation temporaire
- Avantages : compression, lecture partielle, types préservés
- 10-100x plus rapides que CSV pour lecture/écriture

```python
# Write to Parquet (compressed, efficient)
df.to_parquet('data.parquet', compression='gzip', engine='pyarrow')

# Read from Parquet (faster than CSV)
df = pd.read_parquet('data.parquet')
# Read specific columns only
df_subset = pd.read_parquet('data.parquet', columns=['col1', 'col2'])
# Feather format (fast serialization)
df.to_feather('data.feather')
df = pd.read_feather('data.feather')
# Comparison with CSV
# CSV: 100 MB, 10s read time
# Parquet: 20 MB, 1s read time
# Feather: 80 MB, 0.5s read time
```

Alternatives : Dask et Polars

- **Dask** : parallélisation de Pandas pour données out-of-core
- **Polars** : DataFrame Rust ultra-rapide, API similaire à Pandas
- Dask : même API que Pandas, calcul lazy, clusters distribués
- Polars : 5-10x plus rapide que Pandas, gestion native du multi-threading

```python
# Dask: parallel Pandas
import dask.dataframe as dd

ddf = dd.read_csv('large_file.csv')
result = ddf.groupby('category')['value'].mean().compute()
# Polars: fast DataFrame library
import polars as pl
df_polars = pl.read_csv('data.csv')
result = (df_polars
    .filter(pl.col('value') > 100)
    .groupby('category')
    .agg(pl.col('value').mean())
)
# Use Polars for heavy computation, convert to Pandas if needed
df_pandas = df_polars.to_pandas()
```

** Matplotlib **

Introduction à Matplotlib

- Matplotlib est la bibliothèque de visualisation de données la plus populaire en Python
- Créée par John Hunter en 2003, inspirée de MATLAB
- Permet de créer des graphiques statiques, animés et interactifs
- Base pour de nombreuses autres bibliothèques : Seaborn, Pandas plotting, etc.

```python
import matplotlib.pyplot as plt
import numpy as np

# Check version
print(f"Matplotlib version: {plt.matplotlib.__version__}")
# Output: Matplotlib version: 3.8.2
```

Installation et configuration

- Installation : `pip install matplotlib`
- Import standard : `import matplotlib.pyplot as plt`
- Configuration du backend pour affichage interactif ou fichiers
- Style par défaut modifiable avec `plt.style.use()`

```python
import matplotlib.pyplot as plt
import numpy as np

# Set style
plt.style.use('seaborn-v0_8-darkgrid')

# Create simple plot
x: np.ndarray = np.linspace(0, 10, 100)
y: np.ndarray = np.sin(x)

plt.plot(x, y)
plt.title('Simple Sine Wave')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.show()
```

Écosystème de visualisation Python

- **Matplotlib** : bibliothèque de base, très flexible, contrôle total
- **Seaborn** : basé sur Matplotlib, graphiques statistiques élégants
- **Plotly** : graphiques interactifs, dashboards web
- **Bokeh** : visualisations interactives pour navigateurs
- **Altair** : grammaire déclarative, basé sur Vega-Lite
- Matplotlib reste la référence pour publications scientifiques

```python
# Matplotlib: full control
import matplotlib.pyplot as plt

# Seaborn: statistical plots
import seaborn as sns

# Plotly: interactive plots
import plotly.express as px

# Each library has its strengths and use cases
```

Graphique linéaire simple

- Fonction `plt.plot()` pour tracer des courbes
- Arguments : x, y, format de ligne, couleur, label
- Affichage avec `plt.show()`
- Sauvegarde avec `plt.savefig()`

```python
import matplotlib.pyplot as plt
import numpy as np

# Data
x: np.ndarray = np.linspace(0, 2 * np.pi, 100)
y: np.ndarray = np.sin(x)

# Create line plot
plt.figure(figsize=(8, 5))
plt.plot(x, y, color='blue', linewidth=2, label='sin(x)')
plt.title('Sine Function')
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.legend()
plt.grid(True)
plt.show()
```

Plusieurs courbes sur un graphique

- Appels multiples à `plt.plot()` pour superposer des courbes
- Utilisation de `label` pour identifier chaque courbe
- Légende avec `plt.legend()`
- Personnalisation : couleurs, styles de ligne, marqueurs

```python
import matplotlib.pyplot as plt
import numpy as np

x: np.ndarray = np.linspace(0, 2 * np.pi, 100)

plt.figure(figsize=(10, 6))
plt.plot(x, np.sin(x), 'b-', label='sin(x)', linewidth=2)
plt.plot(x, np.cos(x), 'r--', label='cos(x)', linewidth=2)
plt.plot(x, np.sin(2*x), 'g:', label='sin(2x)', linewidth=2)

plt.title('Trigonometric Functions')
plt.xlabel('x')
plt.ylabel('y')
plt.legend(loc='upper right')
plt.grid(True, alpha=0.3)
plt.show()
```

Styles et formats de ligne

- Format : chaîne combinant couleur, style et marqueur (ex: 'ro–')
- Couleurs : 'r' (red), 'g' (green), 'b' (blue), codes hexadécimaux
- Styles : '-' (solid), '–' (dashed), ':' (dotted), '-.' (dash-dot)
- Marqueurs : 'o', 's', '", '*', '+', 'x', etc.

```python
import matplotlib.pyplot as plt
import numpy as np

x: np.ndarray = np.linspace(0, 10, 20)

plt.figure(figsize=(10, 6))
plt.plot(x, x, 'ro-', label='Red circles', markersize=8)
plt.plot(x, x**1.5, 'gs--', label='Green squares', markersize=8)
plt.plot(x, x**2, 'b^:', label='Blue triangles', markersize=8)
plt.plot(x, x**2.5, color='purple', linestyle='-.',
         marker='*', markersize=10, label='Purple stars')

plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Nuage de points avec scatter

- Fonction `plt.scatter()` pour nuages de points
- Paramètres : x, y, s (taille), c (couleur), alpha (transparence)
- Utile pour visualiser corrélations et distributions
- Possibilité de mapper taille et couleur aux données

```python
import matplotlib.pyplot as plt
import numpy as np

# Random data
np.random.seed(42)
n: int = 100
x: np.ndarray = np.random.randn(n)
y: np.ndarray = 2 * x + np.random.randn(n)

plt.figure(figsize=(8, 6))
plt.scatter(x, y, s=50, c='blue', alpha=0.6, edgecolors='black')
plt.title('Scatter Plot Example')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.grid(True, alpha=0.3)
plt.show()
```

Nuage de points avec couleurs et tailles variables

- Paramètre `c` : mapping des couleurs selon valeurs
- Paramètre `s` : taille variable des points
- `plt.colorbar()` pour afficher l'échelle de couleurs
- Visualisation multi-dimensionnelle (x, y, taille, couleur)

```python
import matplotlib.pyplot as plt
import numpy as np

n: int = 200
x: np.ndarray = np.random.randn(n)
y: np.ndarray = np.random.randn(n)
colors: np.ndarray = np.random.rand(n)
sizes: np.ndarray = 1000 * np.random.rand(n)

plt.figure(figsize=(10, 7))
scatter = plt.scatter(x, y, s=sizes, c=colors, alpha=0.5, cmap='viridis', edgecolors='black', linewidth=0.5)
plt.colorbar(scatter, label='Color scale')
plt.title('Scatter Plot with Variable Sizes and Colors')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.show()
```

Scatter plot pour analyse de corrélation

- Ajout d'une ligne de tendance avec `np.polyfit()`
- Visualisation de la relation entre variables
- Calcul du coefficient de corrélation
- Annotations avec `plt.text()` ou `plt.annotate()`

```python
import matplotlib.pyplot as plt
import numpy as np
np.random.seed(42)
x: np.ndarray = np.linspace(0, 10, 50)
y: np.ndarray = 2.5 * x + 5 + np.random.randn(50) * 3
coefficients: np.ndarray = np.polyfit(x, y, 1) # Fit linear trend
trend_line: np.ndarray = coefficients[0] * x + coefficients[1]
plt.figure(figsize=(10, 6))
plt.scatter(x, y, s=60, alpha=0.6, label='Data points')
plt.plot(x, trend_line, 'r--', linewidth=2, label=f'Trend: y={coefficients[0]:.2f}x+{coefficients[1]:.2f}')
plt.title('Scatter Plot with Trend Line')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Histogramme simple

- Fonction `plt.hist()` pour créer des histogrammes
- Paramètres : data, bins (nombre de barres), range, density
- Visualisation de distributions de données
- Retourne : counts, bins, patches

```python
import matplotlib.pyplot as plt
import numpy as np
# Generate normal distribution data
np.random.seed(42)
data: np.ndarray = np.random.randn(1000)
plt.figure(figsize=(10, 6))
n, bins, patches = plt.hist(data, bins=30, color='skyblue', edgecolor='black', alpha=0.7)
plt.title('Histogram of Normal Distribution')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(True, alpha=0.3, axis='y')
plt.show()
```

Histogrammes multiples et comparaison

- Superposition d'histogrammes avec transparence (alpha)
- Paramètre `density=True` pour normaliser
- Comparaison de distributions
- Utilisation de différentes couleurs pour distinction

```python
import matplotlib.pyplot as plt
import numpy as np
np.random.seed(42)
data1: np.ndarray = np.random.normal(0, 1, 1000)
data2: np.ndarray = np.random.normal(2, 1.5, 1000)
plt.figure(figsize=(10, 6))
plt.hist(data1, bins=30, alpha=0.6, color='blue', label='Distribution 1', edgecolor='black')
plt.hist(data2, bins=30, alpha=0.6, color='red', label='Distribution 2', edgecolor='black')
plt.title('Comparison of Two Distributions')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.legend()
plt.grid(True, alpha=0.3, axis='y')
plt.show()
```

Histogramme 2D et densité

- `plt.hist2d()` pour histogrammes bidimensionnels
- Visualisation de densité de points dans le plan
- Paramètre `bins` pour résolution de la grille
- Colorbar pour indiquer la densité

```python
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
n: int = 10000
x: np.ndarray = np.random.randn(n)
y: np.ndarray = np.random.randn(n)

plt.figure(figsize=(10, 8))
h = plt.hist2d(x, y, bins=50, cmap='YlOrRd')
plt.colorbar(h[3], label='Count')
plt.title('2D Histogram (Density Plot)')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.show()
```

Créer plusieurs graphiques avec subplots

- Fonction `plt.subplots()` pour créer une grille de graphiques
- Retourne : figure et array d'axes
- Organisation en lignes et colonnes
- Partage d'axes avec `sharex`, `sharey`

```python
import matplotlib.pyplot as plt
import numpy as np
x: np.ndarray = np.linspace(0, 2 * np.pi, 100)
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes[0, 0].plot(x, np.sin(x))
axes[0, 0].set_title('sin(x)')
axes[0, 1].plot(x, np.cos(x), 'r')
axes[0, 1].set_title('cos(x)')
axes[1, 0].plot(x, np.tan(x))
axes[1, 0].set_title('tan(x)')
axes[1, 0].set_ylim(-5, 5)
axes[1, 1].plot(x, np.exp(-x))
axes[1, 1].set_title('exp(-x)')
plt.tight_layout()
plt.show()
```

Subplots avec axes partagés

- `sharex=True` : même échelle pour l'axe X
- `sharey=True` : même échelle pour l'axe Y
- Facilite la comparaison entre graphiques
- `tight_layout()` pour ajuster automatiquement l'espacement

```python
import matplotlib.pyplot as plt
import numpy as np
x: np.ndarray = np.linspace(0, 10, 100)
data1: np.ndarray = np.sin(x)
data2: np.ndarray = np.cos(x)
data3: np.ndarray = np.sin(2 * x)
fig, axes = plt.subplots(3, 1, figsize=(10, 8), sharex=True)
axes[0].plot(x, data1, 'b-')
axes[0].set_ylabel('sin(x)')
axes[0].grid(True, alpha=0.3)
axes[1].plot(x, data2, 'r-')
axes[1].set_ylabel('cos(x)')
axes[1].grid(True, alpha=0.3)
axes[2].plot(x, data3, 'g-')
axes[2].set_ylabel('sin(2x)')
axes[2].set_xlabel('x')
axes[2].grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Layouts personnalisés avec GridSpec

- `GridSpec` pour layouts complexes et asymétriques
- Spécification flexible de taille et position des sous-graphiques
- Graphiques pouvant s'étendre sur plusieurs cellules
- Plus de contrôle qu'avec `subplots()` simple

```python
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np
fig = plt.figure(figsize=(12, 8))
gs = gridspec.GridSpec(3, 3, figure=fig)
# Large plot spanning 2x2
ax1 = fig.add_subplot(gs[0:2, 0:2])
ax1.plot(np.random.randn(100).cumsum())
ax1.set_title('Large Plot')
# Small plots
ax2 = fig.add_subplot(gs[0, 2])
ax2.hist(np.random.randn(100), bins=20)
ax2.set_title('Plot 2')
ax3 = fig.add_subplot(gs[1, 2])
ax3.scatter(np.random.randn(50), np.random.randn(50))
ax3.set_title('Plot 3')
ax4 = fig.add_subplot(gs[2, :])
ax4.plot(np.random.randn(100))
ax4.set_title('Bottom Plot')
```

Introduction aux graphiques 3D

- Module `mpl_toolkits.mplot3d` pour visualisations 3D
- Projection 3D avec `projection='3d'`
- Types : lignes, surfaces, scatter plots, wireframes
- Rotation interactive dans Jupyter avec widgets

```python
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
# Create 3D axis
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
# Simple 3D line
t: np.ndarray = np.linspace(0, 4 * np.pi, 100)
x: np.ndarray = np.sin(t)
y: np.ndarray = np.cos(t)
z: np.ndarray = t
ax.plot(x, y, z, linewidth=2)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('3D Helix')
plt.show()
```

Nuage de points 3D

- Méthode `ax.scatter()` pour points 3D
- Paramètres : x, y, z, c (couleur), s (taille), marker
- Visualisation de données multi-dimensionnelles
- Angles de vue avec `ax.view_init()`

```python
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
n: int = 200
x: np.ndarray = np.random.randn(n)
y: np.ndarray = np.random.randn(n)
z: np.ndarray = np.random.randn(n)
colors: np.ndarray = z
scatter = ax.scatter(x, y, z, c=colors, cmap='viridis', s=50, alpha=0.6, edgecolors='black')
ax.set_xlabel('X axis')
ax.set_ylabel('Y axis')
ax.set_zlabel('Z axis')
ax.set_title('3D Scatter Plot')
plt.colorbar(scatter)
plt.show()
```

Surfaces 3D

- `ax.plot_surface()` pour surfaces 3D
- Création de grille avec `np.meshgrid()`
- Paramètres : cmap (colormap), alpha, edgecolor
- Visualisation de fonctions $z = f(x, y)$

```python
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')
# Create mesh grid
x: np.ndarray = np.linspace(-5, 5, 100)
y: np.ndarray = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z: np.ndarray = np.sin(np.sqrt(X**2 + Y**2))
# Plot surface
surf = ax.plot_surface(X, Y, Z, cmap='coolwarm', alpha=0.8, edgecolor='none')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('3D Surface: z = sin(sqrt(x² + y²))')
fig.colorbar(surf, shrink=0.5)
plt.show()
```

Graphiques interactifs avec widgets

- Magic command `%matplotlib notebook` pour interactivité
- Zoom, pan, rotation pour graphiques 3D
- Widgets ipywidgets pour contrôles dynamiques
- Backend `widget` ou `notebook`

```python
# In Jupyter notebook
# %matplotlib notebook

import matplotlib.pyplot as plt
import numpy as np

x: np.ndarray = np.linspace(0, 10, 100)

fig, ax = plt.subplots(figsize=(10, 6))
line, = ax.plot(x, np.sin(x))

ax.set_title('Interactive Plot - Zoom and Pan enabled')
ax.set_xlabel('X')
ax.set_ylabel('sin(X)')
ax.grid(True, alpha=0.3)

# Plot is now interactive in Jupyter notebook
# Can zoom, pan, and save
plt.show()
```

Widgets pour paramètres dynamiques

- Module `ipywidgets` pour contrôles interactifs
- Sliders, boutons, dropdowns pour modifier paramètres
- Fonction `interact()` pour liaison automatique
- Mise à jour en temps réel des graphiques

```python
# In Jupyter notebook
from ipywidgets import interact
import matplotlib.pyplot as plt
import numpy as np

def plot_sine(frequency: float = 1.0, amplitude: float = 1.0):
    """Plot sine wave with adjustable parameters."""
    x: np.ndarray = np.linspace(0, 2 * np.pi, 100)
    y: np.ndarray = amplitude * np.sin(frequency * x)

    plt.figure(figsize=(10, 6))
    plt.plot(x, y, linewidth=2)
    plt.title(f'Sine Wave: f={frequency}, A={amplitude}')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.grid(True, alpha=0.3)
    plt.ylim(-3, 3)
    plt.show()
```

Gestion des événements

- Connexion d'événements : clics, mouvements souris, touches clavier
- Méthode `fig.canvas.mpl_connect()`
- Événements : 'button_press_event', 'key_press_event', etc.
- Callbacks pour réactions personnalisées

```python
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(10, 6))
points = []

def on_click(event):
    """Handle mouse click events."""
    if event.inaxes != ax:
        return
    points.append((event.xdata, event.ydata))
    ax.plot(event.xdata, event.ydata, 'ro')
    fig.canvas.draw()

# Connect click event
fig.canvas.mpl_connect('button_press_event', on_click)

ax.set_xlim(0, 10)
ax.set_ylim(0, 10)
```

Modifier les graduations

- `ax.set_xticks()` et `ax.set_yticks()` pour positions
- `ax.set_xticklabels()` pour labels personnalisés
- Formatage avec `matplotlib.ticker`
- Rotation des labels pour lisibilité

```python
import matplotlib.pyplot as plt
import numpy as np

x: np.ndarray = np.linspace(0, 2 * np.pi, 100)
y: np.ndarray = np.sin(x)

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, y)

# Custom tick positions
tick_positions: np.ndarray = np.array([0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi])
tick_labels: list = ['0', 'π/2', 'π', 'π3/2', 'π2']

ax.set_xticks(tick_positions)
ax.set_xticklabels(tick_labels)
ax.set_title('Sine Wave with Custom Ticks')
ax.set_xlabel('x')
ax.set_ylabel('sin(x)')
ax.grid(True, alpha=0.3)
```

Formateurs et localisateurs

- `matplotlib.ticker` pour formatage avancé
- `MultipleLocator` : graduations à intervalles réguliers
- `FuncFormatter` : formatage personnalisé
- `LogLocator`, `AutoLocator` pour échelles spécifiques

```python
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import numpy as np

x: np.ndarray = np.linspace(0, 100, 100)
y: np.ndarray = x ** 2

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, y)

# Set major ticks every 20 units
ax.xaxis.set_major_locator(ticker.MultipleLocator(20))
ax.xaxis.set_minor_locator(ticker.MultipleLocator(5))

# Format y-axis with thousands separator
ax.yaxis.set_major_formatter(ticker.FuncFormatter(lambda x, p: f'{x:,.0f}'))

ax.set_title('Custom Tick Formatting')
ax.set_xlabel('x')
```

Définir les limites des axes

- `ax.set_xlim()` et `ax.set_ylim()` pour limites
- `ax.axis()` pour définir [xmin, xmax, ymin, ymax]
- Ajustement automatique avec `ax.autoscale()`
- Marges avec `ax.margins()`

```python
import matplotlib.pyplot as plt
import numpy as np
x: np.ndarray = np.linspace(0, 10, 100)
y: np.ndarray = np.sin(x)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# Default limits
axes[0].plot(x, y)
axes[0].set_title('Default Limits')
axes[0].grid(True, alpha=0.3)
# Custom limits
axes[1].plot(x, y)
axes[1].set_xlim(2, 8)
axes[1].set_ylim(-0.5, 0.5)
axes[1].set_title('Custom Limits')
axes[1].grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Échelles logarithmiques

- `ax.set_xscale('log')` pour échelle logarithmique en X
- `ax.set_yscale('log')` pour échelle logarithmique en Y
- Utile pour données couvrant plusieurs ordres de grandeur
- Aussi : 'linear', 'symlog', 'logit'

```python
import matplotlib.pyplot as plt
import numpy as np
x: np.ndarray = np.logspace(0, 3, 100)
y: np.ndarray = x ** 2
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# Linear scale
axes[0].plot(x, y)
axes[0].set_title('Linear Scale')
axes[0].grid(True, alpha=0.3)
# Log-log scale
axes[1].plot(x, y)
axes[1].set_xscale('log')
axes[1].set_yscale('log')
axes[1].set_title('Log-Log Scale')
axes[1].grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.show()
```

Graphiques directement depuis DataFrame

- Méthode `df.plot()` pour graphiques rapides
- Types : 'line', 'bar', 'barh', 'hist', 'box', 'scatter', 'area'
- Index utilisé automatiquement pour axe X
- Intégration native avec Matplotlib

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
dates = pd.date_range('2024-01-01', periods=100, freq='D')
df = pd.DataFrame({
    'A': np.random.randn(100).cumsum(),
    'B': np.random.randn(100).cumsum(),
    'C': np.random.randn(100).cumsum()
}, index=dates)
# Plot directly from DataFrame
df.plot(figsize=(12, 6), linewidth=2, alpha=0.7)
plt.title('Time Series Data')
plt.xlabel('Date')
plt.ylabel('Value')
plt.grid(True, alpha=0.3)
plt.legend(loc='best')
plt.show()
```

Différents types de graphiques

- `df.plot.bar()` : graphique en barres
- `df.plot.scatter()` : nuage de points
- `df.plot.hist()` : histogrammes
- `df.plot.box()` : boîtes à moustaches

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Create sample data
df = pd.DataFrame({
    'Category': ['A', 'B', 'C', 'D', 'E'],
    'Value1': [23, 45, 56, 78, 32],
    'Value2': [38, 52, 61, 44, 29]
})
df.set_index('Category').plot.bar(figsize=(10, 6), rot=0) # Bar plot
plt.title('Comparison by Category')
plt.xlabel('Category')
plt.ylabel('Values')
plt.legend(title='Metrics')
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

Groupby et visualisation

- Combinaison de `groupby()` avec `plot()`
- Agrégations visuelles par catégories
- Graphiques empilés avec `stacked=True`
- Sous-graphiques avec `subplots=True`

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar', 'Jan', 'Feb', 'Mar'] * 2,
    'Region': ['North']*6 + ['South']*6,
    'Sales': [100, 120, 140, 110, 130, 150, 90, 95, 105, 85, 100, 110]
})
# Group and plot
grouped = df.groupby(['Month', 'Region'])['Sales'].sum().unstack()
grouped.plot.bar(figsize=(10, 6), rot=0)
plt.title('Sales by Month and Region')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.legend(title='Region')
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

Matrice de corrélation et heatmaps

- Visualisation des corrélations avec `df.corr()`
- Heatmap avec `plt.imshow()` ou Seaborn
- Annotations pour valeurs exactes et choix de colormap appropriée

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Create correlated data
np.random.seed(42)
df = pd.DataFrame({ 'A': np.random.randn(100), 'B': np.random.randn(100), 'C': np.random.randn(100), 'D':
    np.random.randn(100) })
df['E'] = df['A'] + np.random.randn(100) * 0.5
corr_matrix = df.corr() # Compute and plot correlation matrix
fig, ax = plt.subplots(figsize=(10, 8))
im = ax.imshow(corr_matrix, cmap='coolwarm', vmin=-1, vmax=1)
ax.set_xticks(np.arange(len(corr_matrix.columns)))
ax.set_yticks(np.arange(len(corr_matrix.columns)))
ax.set_xticklabels(corr_matrix.columns)
ax.set_yticklabels(corr_matrix.columns)
plt.colorbar(im, label='Correlation')
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
```

Séries temporelles avec Pandas

- Graphiques de séries temporelles avec resampling
- Moyennes mobiles et tendances
- Gestion automatique des dates sur l'axe X
- Annotations d'événements spécifiques

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
dates = pd.date_range('2023-01-01', periods=365, freq='D') # Create time series
ts = pd.Series(np.random.randn(365).cumsum(), index=dates)
ma_30 = ts.rolling(window=30).mean() # Calculate moving average
plt.figure(figsize=(14, 6))
plt.plot(ts.index, ts.values, label='Daily data', alpha=0.5)
plt.plot(ma_30.index, ma_30.values, label='30-day MA', linewidth=2, color='red')
plt.title('Time Series with Moving Average')
plt.xlabel('Date')
plt.ylabel('Value')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

** SciPy **

Introduction à SciPy

- La bibliothèque SciPy est une extension de NumPy dédiée au calcul scientifique avancé.
- SciPy fournit des outils pour l'optimisation, l'intégration, l'algèbre linéaire, les statistiques et le traitement du signal.
- Elle repose sur NumPy et exploite des structures de données efficaces pour les calculs numériques.
- SciPy est largement utilisée en ingénierie, physique, mathématiques appliquées et data science.

Installation de SciPy

- SciPy peut être installée avec `pip install scipy`.
- Il est recommandé d'utiliser un environnement virtuel pour isoler les dépendances.
- Vérification de l'installation :

```python
import scipy as sp
import numpy as np

# Check version
print(f"SciPy version: {sp.__version__}")
# Output: SciPy version: 1.11.4
```

Modules principaux de SciPy

- `scipy.integrate` : intégration numérique et résolution d'équations différentielles
- `scipy.optimize` : optimisation et recherche de racines
- `scipy.linalg` : algèbre linéaire avancée
- `scipy.stats` : statistiques et distributions de probabilité
- `scipy.interpolate` : interpolation et ajustement de courbes
- `scipy.fft` : transformée de Fourier rapide
- `scipy.signal` : traitement du signal
- `scipy.sparse` : matrices creuses

Intégration numérique avec SciPy

- SciPy propose des méthodes d'intégration numérique via `scipy.integrate`.
- Exemple : calcul d'une intégrale définie

$$\int_a^b f(x)\,dx$$

- Fonction principale : `quad` pour l'intégration adaptative.
- Retourne le résultat et l'estimation de l'erreur.

```python
from scipy import integrate
from typing import Callable
def quadratic_function(x: float) -> float:
    """Compute x squared."""
    return x ** 2
result, error = integrate.quad(quadratic_function, 0, 1)
print(f"Result: {result:.6f}, Error: {error:.2e}")
# Output: Result: 0.333333, Error: 3.70e-15
```

Intégration de fonctions multiples

- Intégration de fonctions trigonométriques ou transcendantes.
- Gestion automatique de la précision et du nombre de subdivisions.
- Possibilité de spécifier des limites infinies.

```python
import numpy as np
from scipy import integrate

def sin_function(x: float) -> float:
    """Compute sine of x."""
    return np.sin(x)

# Integration from 0 to pi
result: float
error: float
result, error = integrate.quad(sin_function, 0, np.pi)

print(f"Integral of sin(x) from 0 to pi: {result:.6f}")
# Output: Integral of sin(x) from 0 to pi: 2.000000
```

Intégration double et triple

- `dblquad` : intégration double sur un domaine rectangulaire ou variable.
- `tplquad` : intégration triple pour les volumes.
- Permet de calculer des aires, volumes, et centres de masse.

```python
from scipy import integrate
from typing import Callable

def integrand(y: float, x: float) -> float:
    """Compute x*y as integrand."""
    return x * y

# Double integral: integrate y from 0 to 1, then x from 0 to 2
result: float
error: float
result, error = integrate.dblquad(integrand, 0, 2, 0, 1)

print(f"Double integral result: {result:.6f}")
# Output: Double integral result: 1.000000
```

Optimisation avec SciPy

- Module : `scipy.optimize`
- Permet de trouver les minima et maxima de fonctions.
- Algorithmes : gradient conjugué, Nelder-Mead, BFGS, L-BFGS-B.
- Fonction principale : `minimize` pour l'optimisation générale.

```python
from scipy.optimize import minimize
import numpy as np
from typing import Callable

def rosenbrock(x: np.ndarray) -> float:
    """Rosenbrock function for optimization testing."""
    return (1 - x[0]) ** 2 + 100 * (x[1] - x[0] ** 2) ** 2

# Initial guess
x0: np.ndarray = np.array([0.0, 0.0])

# Minimize the function
result = minimize(rosenbrock, x0, method='BFGS')

print(f"Minimum at: {result.x}")
# Output: Minimum at: [1. 1.]
```

Recherche de racines

- Fonction `root_scalar` pour trouver les zéros d'une fonction scalaire.
- Méthodes disponibles : bisection, Newton, Brent, Ridder.
- Utile pour résoudre des équations non linéaires.

```python
from scipy.optimize import root_scalar
from typing import Callable

def equation(x: float) -> float:
    """Equation to solve: x^3 - 2x - 5 = 0."""
    return x ** 3 - 2 * x - 5

# Find root using brentq method
result = root_scalar(equation, bracket=[2, 3], method='brentq')

print(f"Root found at x = {result.root:.6f}")
# Output: Root found at x = 2.094551
```

Optimisation avec contraintes

- Support des contraintes linéaires et non linéaires.
- Méthodes : SLSQP (Sequential Least Squares Programming), trust-constr.
- Définition de bornes pour les variables.

```python
from scipy.optimize import minimize
import numpy as np

def objective(x: np.ndarray) -> float:
    """Objective function to minimize."""
    return x[0] ** 2 + x[1] ** 2


bounds = [(0, 2), (0, 2)] # Bounds: x[0] in [0, 2], x[1] in [0, 2]

# Constraint: x[0] + x[1] >= 1
constraints = {'type': 'ineq', 'fun': lambda x: x[0] + x[1] - 1}

result = minimize(objective, [1, 1], method='SLSQP', bounds=bounds, constraints=constraints)
print(f"Solution: {result.x}")
# Output: Solution: [0.5 0.5]
```

Algèbre linéaire avec SciPy

- Module : `scipy.linalg`
- Permet de résoudre des systèmes linéaires :

$$Ax = b$$

- Calcul d'inverse, déterminant, décompositions matricielles.
- Plus complet que `numpy.linalg`.

```python
import numpy as np
from scipy import linalg

# System: Ax = b
A: np.ndarray = np.array([[3, 2], [1, 4]], dtype=float)
b: np.ndarray = np.array([7, 6], dtype=float)

# Solve the system
x: np.ndarray = linalg.solve(A, b)

print(f"Solution: x = {x}")
# Output: Solution: x = [1.8 0.95]
```

Décompositions matricielles

- `LU` : décomposition LU pour résolution efficace de systèmes multiples.
- `QR` : décomposition QR pour les moindres carrés.
- `SVD` : décomposition en valeurs singulières.
- `Cholesky` : pour les matrices symétriques définies positives.

```python
import numpy as np
from scipy import linalg

# Matrix for decomposition
A: np.ndarray = np.array([[4, 2], [2, 3]], dtype=float)

# LU decomposition
P: np.ndarray
L: np.ndarray
U: np.ndarray
P, L, U = linalg.lu(A)

print(f"L matrix:\n{L}")
print(f"U matrix:\n{U}")
# Verification: P @ A = L @ U
```

Valeurs et vecteurs propres

- Calcul des valeurs propres (eigenvalues) et vecteurs propres (eigenvectors).
- Fonction `eig` pour les matrices générales.
- Fonction `eigh` pour les matrices hermitiennes (plus rapide).

```python
import numpy as np
from scipy import linalg

# Symmetric matrix
A: np.ndarray = np.array([[4, 1], [1, 3]], dtype=float)

# Compute eigenvalues and eigenvectors
eigenvalues: np.ndarray
eigenvectors: np.ndarray
eigenvalues, eigenvectors = linalg.eigh(A)

print(f"Eigenvalues: {eigenvalues}")
print(f"Eigenvectors:\n{eigenvectors}")
# Output: Eigenvalues: [2.38196601 4.61803399]
```

Distributions statistiques

- Module : `scipy.stats`
- Plus de 100 distributions continues et discrètes.
- Fonctions : PDF, CDF, PPF (inverse de CDF), génération aléatoire.
- Support des tests statistiques et des estimations de paramètres.

```python
from scipy import stats
import numpy as np

# Normal distribution with mean=0, std=1
dist = stats.norm(loc=0, scale=1)
# Probability density function at x=0
pdf_value: float = dist.pdf(0)
# Cumulative distribution function at x=1
cdf_value: float = dist.cdf(1)
# Generate random samples
samples: np.ndarray = dist.rvs(size=1000, random_state=42)
print(f"PDF at x=0: {pdf_value:.6f}")
print(f"CDF at x=1: {cdf_value:.6f}")
# Output: PDF at x=0: 0.398942, CDF at x=1: 0.841345
```

Tests statistiques

- Test t de Student : `ttest_ind`, `ttest_rel`
- Test de normalité : `shapiro`, `normaltest`
- Test du chi-deux : `chisquare`
- Corrélation : `pearsonr`, `spearmanr`

```python
from scipy import stats
import numpy as np

# Two samples for comparison
sample1: np.ndarray = np.array([2.3, 2.5, 2.8, 3.0, 3.2])
sample2: np.ndarray = np.array([3.1, 3.4, 3.6, 3.9, 4.1])

# Independent t-test
statistic: float
p_value: float
statistic, p_value = stats.ttest_ind(sample1, sample2)

print(f"t-statistic: {statistic:.4f}")
print(f"p-value: {p_value:.4f}")
# Output: t-statistic: -4.7958, p-value: 0.0010
```

Analyse descriptive

- Fonction `describe` pour statistiques descriptives complètes.
- Calcul automatique : taille, moyenne, variance, skewness, kurtosis.
- Minimum, maximum et étendue des données.

```python
from scipy import stats
import numpy as np

# Sample data
data: np.ndarray = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Descriptive statistics
description = stats.describe(data)

print(f"Mean: {description.mean:.2f}")
print(f"Variance: {description.variance:.2f}")
print(f"Skewness: {description.skewness:.4f}")
print(f"Kurtosis: {description.kurtosis:.4f}")
# Output: Mean: 5.50, Variance: 9.17
```

Transformée de Fourier avec SciPy

- Module : `scipy.fft`
- Permet de passer du domaine temporel au domaine fréquentiel.
- Transformée de Fourier discrète (DFT) :

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2i\pi kn/N}$$

- Algorithme FFT (Fast Fourier Transform) pour calcul efficace.

```python
from scipy.fft import fft, fftfreq
import numpy as np
sampling_rate: int = 100 # Signal: cosine wave
duration: float = 1.0
t: np.ndarray = np.linspace(0, duration, sampling_rate)
signal: np.ndarray = np.cos(2 * np.pi * 5 * t)
fft_values: np.ndarray = fft(signal) # Compute FFT
frequencies: np.ndarray = fftfreq(len(signal), 1/sampling_rate)
print(f"FFT computed with {len(fft_values)} points")
# Output: FFT computed with 100 points
```

Transformée de Fourier inverse

- Fonction `ifft` pour la transformée inverse.
- Permet de reconstruire le signal temporel depuis le spectre.
- Vérification : `ifft(fft(x))` doit retrouver `x`.

```python
from scipy.fft import fft, ifft
import numpy as np

# Original signal
signal: np.ndarray = np.array([1.0, 2.0, 3.0, 4.0])

# Forward FFT
fft_values: np.ndarray = fft(signal)

# Inverse FFT
reconstructed: np.ndarray = ifft(fft_values).real

# Check reconstruction
is_equal: bool = np.allclose(signal, reconstructed)

print(f"Reconstruction successful: {is_equal}")
# Output: Reconstruction successful: True
```

Analyse spectrale

- Identification des fréquences dominantes dans un signal.
- Utile pour l'analyse de vibrations, sons, signaux électriques.
- Calcul de l'amplitude du spectre avec `np.abs(fft_values)`.

```python
from scipy.fft import fft, fftfreq
import numpy as np

# Complex signal: sum of two sinusoids
t: np.ndarray = np.linspace(0, 1, 500)
signal: np.ndarray = np.sin(2 * np.pi * 10 * t) + 0.5 * np.sin(2 * np.pi * 20 * t)

# FFT and frequencies
fft_values: np.ndarray = fft(signal)
freqs: np.ndarray = fftfreq(len(signal), t[1] - t[0])

# Amplitude spectrum
amplitude: np.ndarray = np.abs(fft_values)

print(f"Dominant frequencies identified")
# Analysis reveals peaks at 10 Hz and 20 Hz
```

Interpolation avec SciPy

- Module : `scipy.interpolate`
- Permet d'estimer des valeurs intermédiaires entre des points connus.
- Interpolation linéaire : connexion par segments de droite.
- Autres méthodes : cubique, spline, nearest neighbor.

```python
from scipy.interpolate import interp1d
import numpy as np

# Known points
x: np.ndarray = np.array([0, 1, 2, 3])
y: np.ndarray = np.array([0, 1, 4, 9])

# Create linear interpolation function
f = interp1d(x, y, kind='linear')

# Interpolate at x=1.5
interpolated_value: float = float(f(1.5))

print(f"Interpolated value at x=1.5: {interpolated_value}")
# Output: Interpolated value at x=1.5: 2.5
```

Interpolation cubique et splines

- Interpolation cubique : polynômes de degré 3 entre chaque paire de points.
- Splines : courbes lisses passant par tous les points.
- Fonction `CubicSpline` pour interpolation de haute qualité.

```python
from scipy.interpolate import CubicSpline
import numpy as np

# Data points
x: np.ndarray = np.array([0, 1, 2, 3, 4])
y: np.ndarray = np.array([0, 2, 1, 3, 2])

# Create cubic spline
cs = CubicSpline(x, y)

# Evaluate at intermediate points
x_new: np.ndarray = np.linspace(0, 4, 100)
y_new: np.ndarray = cs(x_new)

print(f"Spline evaluated at {len(x_new)} points")
# Output: Spline evaluated at 100 points
```

Interpolation 2D

- Interpolation sur grilles 2D avec `interp2d` ou `RectBivariateSpline`.
- Utile pour données géographiques, images, champs de température.
- Supporte différentes méthodes : linéaire, cubique.

```python
from scipy.interpolate import RectBivariateSpline
import numpy as np

# Grid points
x: np.ndarray = np.array([0, 1, 2])
y: np.ndarray = np.array([0, 1, 2])
z: np.ndarray = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Create 2D spline
interp = RectBivariateSpline(x, y, z)

# Evaluate at (1.5, 1.5)
value: float = float(interp(1.5, 1.5))

print(f"Interpolated value at (1.5, 1.5): {value:.2f}")
# Output: Interpolated value at (1.5, 1.5): 5.50
```

Ajustement de courbe avec SciPy

- Module : `scipy.optimize`
- Permet de trouver les paramètres optimaux d'un modèle :

$$y = f(x, \theta)$$

- Fonction : `curve_fit` utilisant les moindres carrés non linéaires.
- Retourne les paramètres optimaux et leur covariance.

```python
from scipy.optimize import curve_fit
import numpy as np
from typing import Callable

def linear_model(x: np.ndarray, a: float, b: float) -> np.ndarray:
    """Linear model: y = a*x + b."""
    return a * x + b

x_data: np.ndarray = np.array([1, 2, 3, 4, 5])
y_data: np.ndarray = np.array([2.1, 4.0, 6.1, 7.9, 10.2])
params, covariance = curve_fit(linear_model, x_data, y_data)
print(f"Parameters: a={params[0]:.4f}, b={params[1]:.4f}")
# Output: Parameters: a=2.0200, b=0.0400
```

Ajustement de modèles non linéaires

- `curve_fit` supporte tous types de fonctions non linéaires.
- Modèles courants : exponentielle, gaussienne, sigmoïde.
- Possibilité de fournir des valeurs initiales pour les paramètres.

```python
from scipy.optimize import curve_fit
import numpy as np

def exponential_model(x: np.ndarray, a: float, b: float,
                      c: float) -> np.ndarray:
    """Exponential decay model."""
    return a * np.exp(-b * x) + c

x_data: np.ndarray = np.linspace(0, 4, 50)
y_data: np.ndarray = 2.5 * np.exp(-1.3 * x_data) + 0.5

# Fit with initial guess
params, _ = curve_fit(exponential_model, x_data, y_data,
                      p0=[2, 1, 0])

print(f"Fitted: a={params[0]:.2f}, b={params[1]:.2f}, c={params[2]:.2f}")
# Output: Fitted: a=2.50, b=1.30, c=0.50
```

Évaluation de la qualité d'ajustement

- Matrice de covariance pour estimer l'incertitude des paramètres.
- Calcul du coefficient de détermination R^2.
- Analyse des résidus pour valider le modèle.

```python
from scipy.optimize import curve_fit
import numpy as np

def model(x: np.ndarray, a: float, b: float) -> np.ndarray:
    """Simple quadratic model."""
    return a * x ** 2 + b

x_data: np.ndarray = np.array([1, 2, 3, 4, 5])
y_data: np.ndarray = np.array([1.1, 4.2, 9.1, 16.0, 25.1])

params, cov = curve_fit(model, x_data, y_data)

# Standard deviation of parameters
param_std: np.ndarray = np.sqrt(np.diag(cov))

print(f"a = {params[0]:.4f} ± {param_std[0]:.4f}")
print(f"b = {params[1]:.4f} ± {param_std[1]:.4f}")
# Output: a = 1.0020 ± 0.0052, b = 0.0400 ± 0.0891
```

Filtrage de signaux

- Module : `scipy.signal`
- Conception et application de filtres numériques.
- Types de filtres : passe-bas, passe-haut, passe-bande, coupe-bande.
- Fonction `butter` pour filtres de Butterworth.

```python
from scipy import signal
import numpy as np

# Design a lowpass Butterworth filter
order: int = 4
cutoff_freq: float = 0.2  # Normalized frequency (0 to 1)

b: np.ndarray
a: np.ndarray
b, a = signal.butter(order, cutoff_freq, btype='low')

# Apply filter to a signal
noisy_signal: np.ndarray = np.random.randn(100)
filtered_signal: np.ndarray = signal.filtfilt(b, a, noisy_signal)

print(f"Signal filtered with order {order} Butterworth filter")
# Output: Signal filtered with order 4 Butterworth filter
```

Détection de pics

- Fonction `find_peaks` pour identifier les maxima locaux.
- Paramètres : hauteur minimale, distance entre pics, proéminence.
- Utile pour analyse de spectres, ECG, données chronologiques.

```python
from scipy import signal
import numpy as np

# Signal with peaks
x: np.ndarray = np.linspace(0, 10, 1000)
signal_data: np.ndarray = np.sin(x) + 0.5 * np.sin(3 * x)

# Find peaks with minimum height of 0.5
peaks: np.ndarray
properties: dict
peaks, properties = signal.find_peaks(signal_data, height=0.5)

print(f"Number of peaks found: {len(peaks)}")
print(f"Peak heights: {properties['peak_heights'][:5]}")
# Output: Number of peaks found: 15
```

Spectrogramme

- Fonction `spectrogram` pour analyse temps-fréquence.
- Visualise comment le spectre évolue dans le temps.
- Paramètres : taille de fenêtre, chevauchement, fenêtre d'apodisation.

```python
from scipy import signal
import numpy as np

# Generate chirp signal (frequency increases with time)
t: np.ndarray = np.linspace(0, 10, 1000)
chirp: np.ndarray = signal.chirp(t, f0=1, f1=10, t1=10, method='linear')

# Compute spectrogram
f: np.ndarray
times: np.ndarray
Sxx: np.ndarray
f, times, Sxx = signal.spectrogram(chirp, fs=100)

print(f"Spectrogram shape: {Sxx.shape}")
print(f"Frequency bins: {len(f)}, Time bins: {len(times)}")
# Output: Spectrogram shape: (129, 9)
```

Résolution d'équations différentielles ordinaires

- Fonction `solve_ivp` pour problèmes à valeur initiale.
- Résolution de systèmes d'EDO :

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

- Méthodes : RK45 (Runge-Kutta), BDF (backward differentiation).

```python
from scipy.integrate import solve_ivp
import numpy as np
from typing import Callable
def exp_decay(t: float, y: np.ndarray) -> np.ndarray:
    """dy/dt = -k*y where k=0.5."""
    return -0.5 * y
# Initial condition: y(0) = 1
t_span: tuple = (0, 10)
y0: np.ndarray = np.array([1.0])
# Solve ODE
solution = solve_ivp(exp_decay, t_span, y0, method='RK45', dense_output=True)
print(f"Solution computed at {len(solution.t)} time points")
# Output: Solution computed at 33 time points
```

Systèmes d'équations différentielles

- Résolution de systèmes couplés (plusieurs variables dépendantes).
- Exemple : équations de Lotka-Volterra (prédateur-proie).
- Le vecteur d'état contient toutes les variables.

```python
from scipy.integrate import solve_ivp
import numpy as np

def lotka_volterra(t: float, z: np.ndarray) -> np.ndarray:
    """Predator-prey model: x=prey, y=predator."""
    x, y = z
    alpha, beta, delta, gamma = 1.5, 1.0, 3.0, 1.0
    dxdt = alpha * x - beta * x * y
    dydt = delta * x * y - gamma * y
    return np.array([dxdt, dydt])

# Initial populations
y0: np.ndarray = np.array([10.0, 5.0])

sol = solve_ivp(lotka_volterra, (0, 15), y0, method='RK45', dense_output=True)
print(f"System solved with {len(sol.t)} evaluations")
# Output: System solved with 89 evaluations
```

Événements et terminaison

- Détection d'événements pendant l'intégration (e.g., croisement de zéro).
- Terminaison anticipée quand une condition est remplie.
- Fonction événement doit retourner 0 au moment de l'événement.

```python
from scipy.integrate import solve_ivp
import numpy as np

def falling_ball(t: float, y: np.ndarray) -> np.ndarray:
    """Free fall: position and velocity."""
    position, velocity = y
    return np.array([velocity, -9.81])

def hit_ground(t: float, y: np.ndarray) -> float:
    """Event: ball hits ground (position = 0)."""
    return y[0]

hit_ground.terminal = True  # Stop integration at event
y0: np.ndarray = np.array([100.0, 0.0])  # Start at 100m, v=0
sol = solve_ivp(falling_ball, (0, 20), y0, events=hit_ground)
print(f"Ball hit ground at t={sol.t_events[0][0]:.2f}s")
# Output: Ball hit ground at t=4.52s
```

Matrices creuses avec SciPy

- Module : `scipy.sparse`
- Stockage efficace de matrices avec beaucoup de zéros.
- Formats : CSR (Compressed Sparse Row), COO (Coordinate), LIL (List of Lists).
- Économie de mémoire et calculs plus rapides pour grandes matrices.

```python
from scipy import sparse
import numpy as np

# Dense matrix with many zeros
dense_matrix: np.ndarray = np.array([
    [1, 0, 0, 2],
    [0, 0, 3, 0],
    [4, 0, 0, 0]
], dtype=float)

# Convert to sparse CSR format
sparse_matrix = sparse.csr_matrix(dense_matrix)
print(f"Dense size: {dense_matrix.nbytes} bytes")
print(f"Sparse size: {sparse_matrix.data.nbytes} bytes")
# Output: Dense size: 96 bytes, Sparse size: 32 bytes
```

Opérations sur matrices creuses

- Addition, multiplication matricielle optimisées pour structures creuses.
- Résolution de systèmes linéaires creux avec `spsolve`.
- Construction directe : `sparse.eye`, `sparse.diags`.

```python
from scipy import sparse
import numpy as np

# Create diagonal sparse matrix
n: int = 1000
diagonals: np.ndarray = np.array([1, 2, 1])
offsets: np.ndarray = np.array([-1, 0, 1])

A = sparse.diags(diagonals, offsets, shape=(n, n), format='csr')
# Create vector
b: np.ndarray = np.random.rand(n)
# Solve sparse system Ax = b
x: np.ndarray = sparse.linalg.spsolve(A, b)
print(f"Sparse system solved for {n}x{n} matrix")
# Output: Sparse system solved for 1000x1000 matrix
```

Applications des matrices creuses

- Graphes et réseaux (matrices d'adjacence).
- Éléments finis et différences finies en ingénierie.
- Traitement d'images et apprentissage automatique.
- Systèmes linéaires de très grande dimension.

```python
from scipy import sparse
import numpy as np

# Create adjacency matrix for a graph (5 nodes)
row: np.ndarray = np.array([0, 0, 1, 2, 2, 3])
col: np.ndarray = np.array([1, 2, 3, 3, 4, 4])
data: np.ndarray = np.ones(6)

adjacency = sparse.coo_matrix((data, (row, col)), shape=(5, 5))

# Convert to CSR for efficient operations
adjacency_csr = adjacency.tocsr()

print(f"Graph with {adjacency.nnz} edges represented")
# Output: Graph with 6 edges represented
```

** Merci de votre attention ! **